



Table of contents

- 1. Linear Regression with One Variable**
- 2. Linear Regression with Multiple Variable**
- 3. Neural Network**
- 4. Convolutional Neural Network**
- 5. Recurrent Neural Network**
- 6. Generative Adversarial Networks**

Linear Regression with One Variable

1. Linear Regression with One Variable Example

Training set

<i>features</i> size in feet ² (x)	<i>targets</i> price \$1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

Model: $f_{w,b}(x) = wx + b$

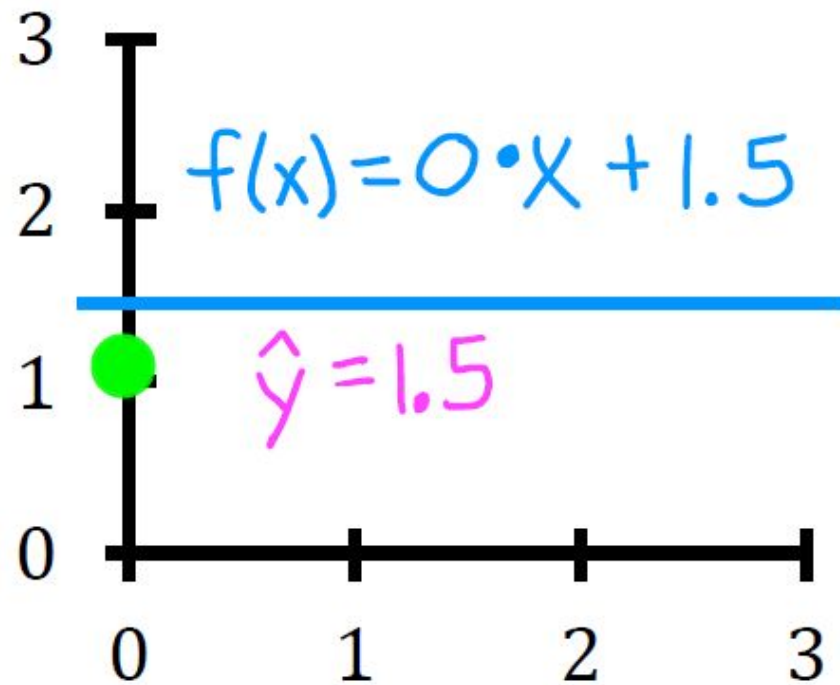
w, b : parameters
coefficients
weights

What do w, b do?

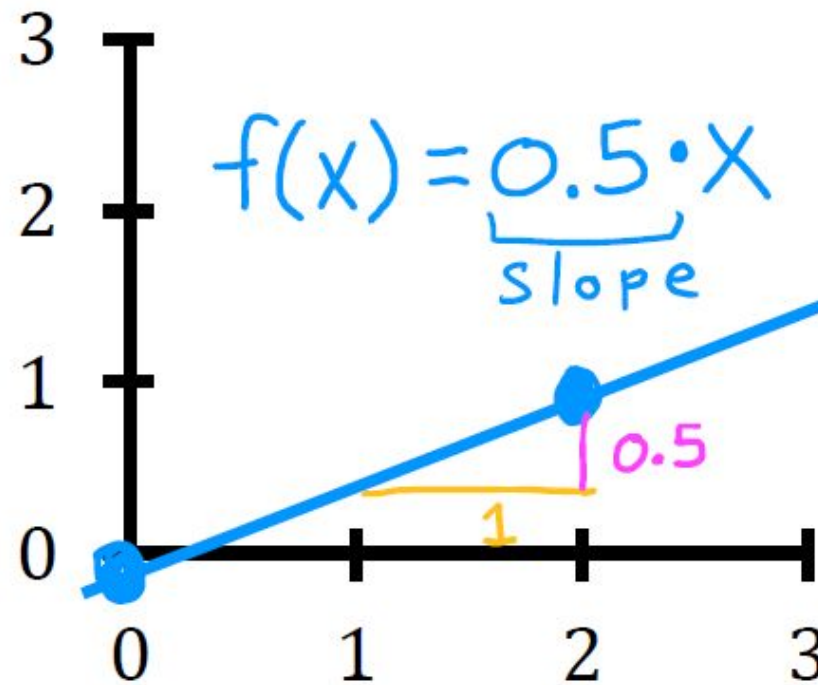
1. Linear Regression with One Variable Example

$$\underline{f_{w,b}}(x) = wx + b$$

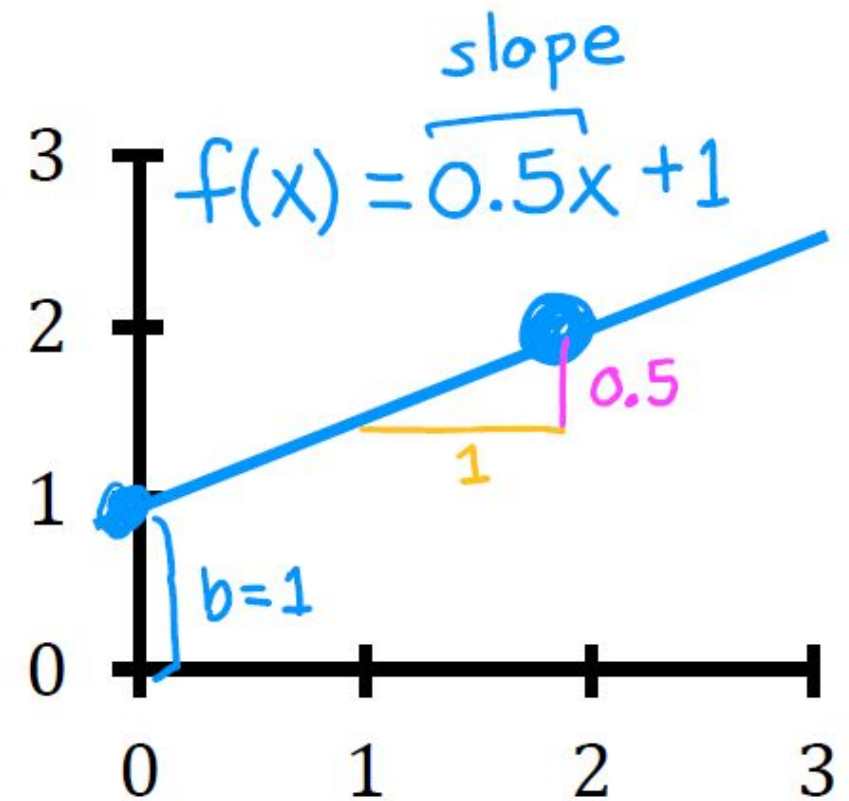
$f(x)$



→ $w = 0$
→ $b = 1.5$

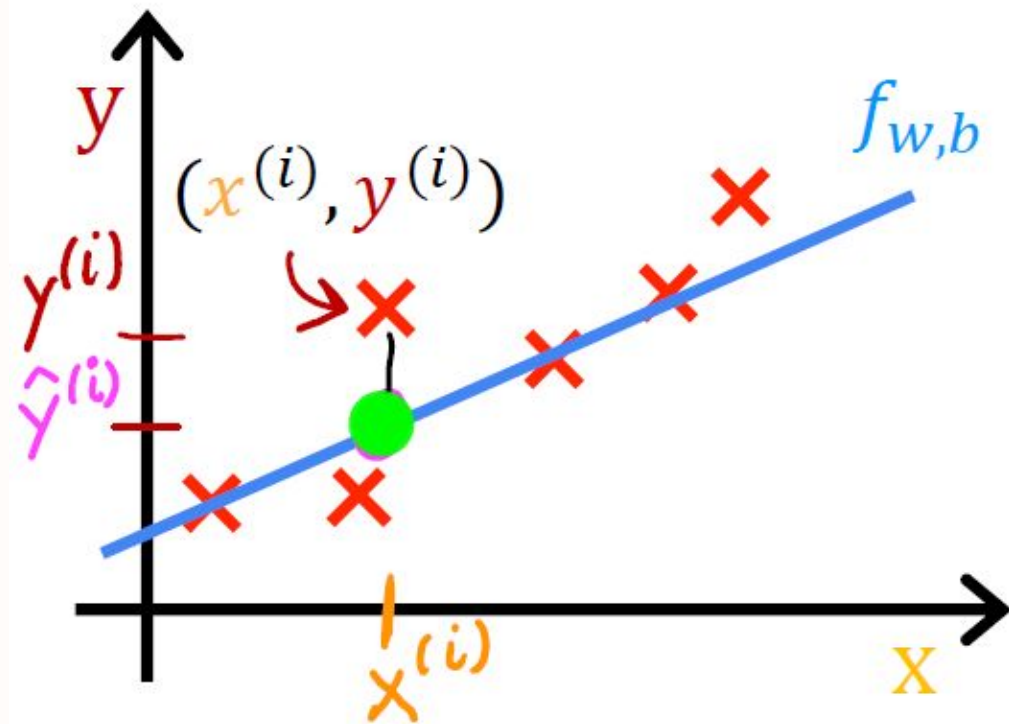


→ $w = 0.5$
→ $b = 0$



→ $w = 0.5$
→ $b = 1$

2. Cost function



$$\hat{y}^{(i)} = f_{w,b}(x^{(i)}) \quad \leftarrow$$

$$f_{w,b}(x^{(i)}) = wx^{(i)} + b$$

Cost function: Squared error cost function

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

error

m = number of training examples

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2$$

intuition (next!)

Find w, b :

$\hat{y}^{(i)}$ is close to $y^{(i)}$ for all $(x^{(i)}, y^{(i)})$.

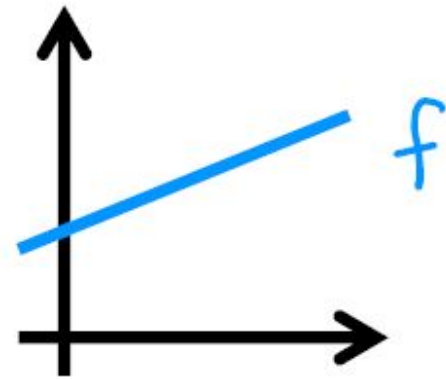
2. Cost function

model:

$$\underline{f_{w,b}(x) = wx + b}$$

parameters:

$$\underline{w, b}$$



cost function:

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

goal:

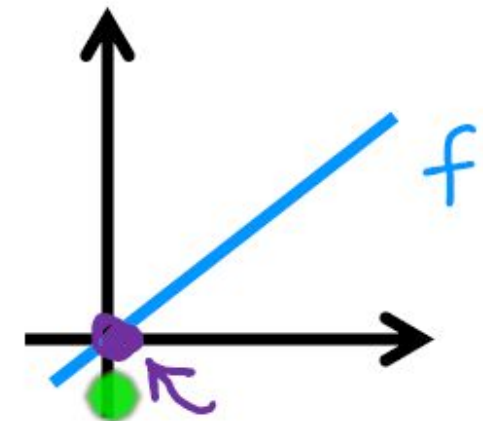
$$\underset{w, b}{\text{minimize}} J(w, b)$$

simplified

$$f_w(x) = \underline{wx}$$

$$b = \emptyset$$

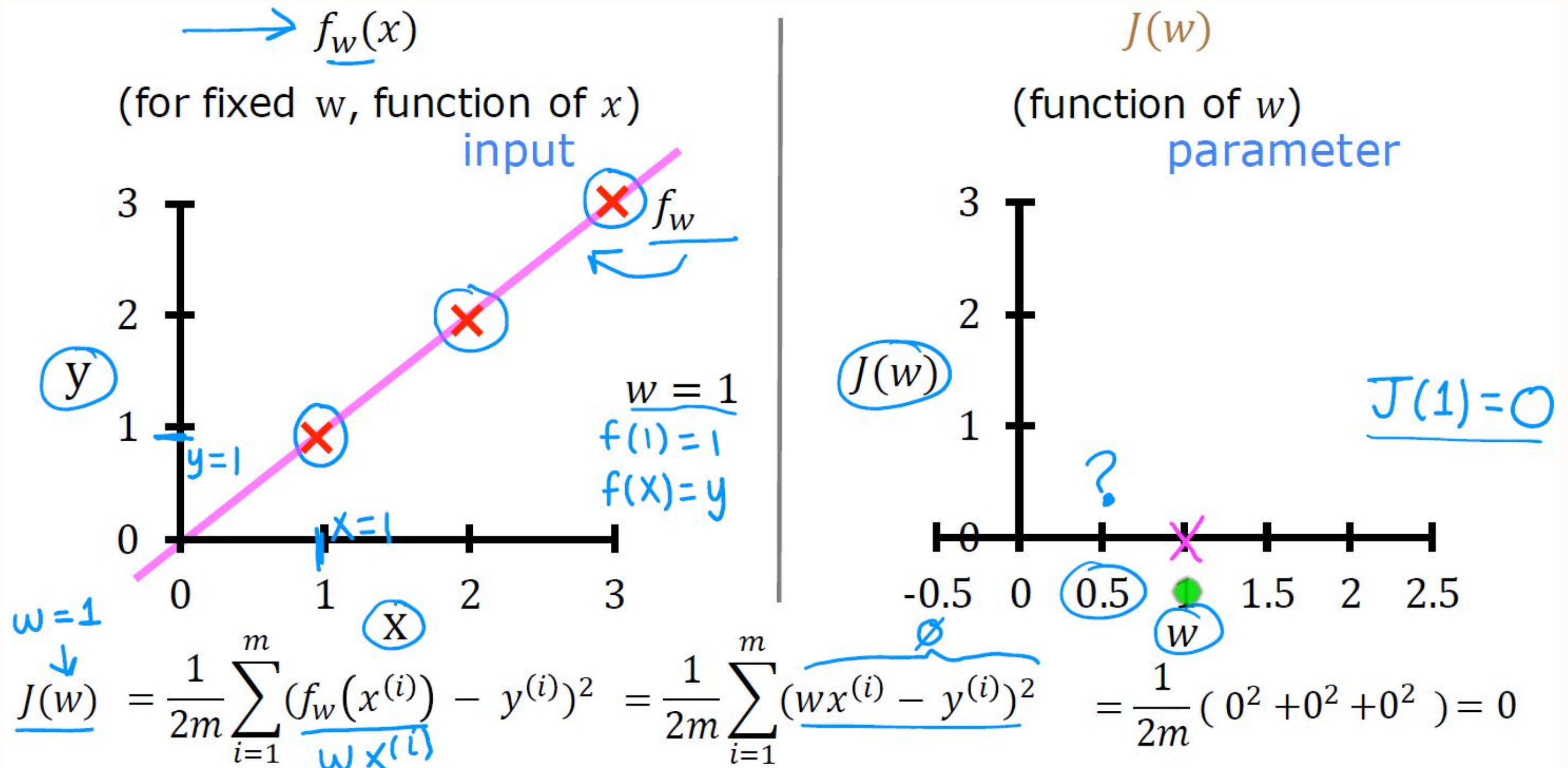
w



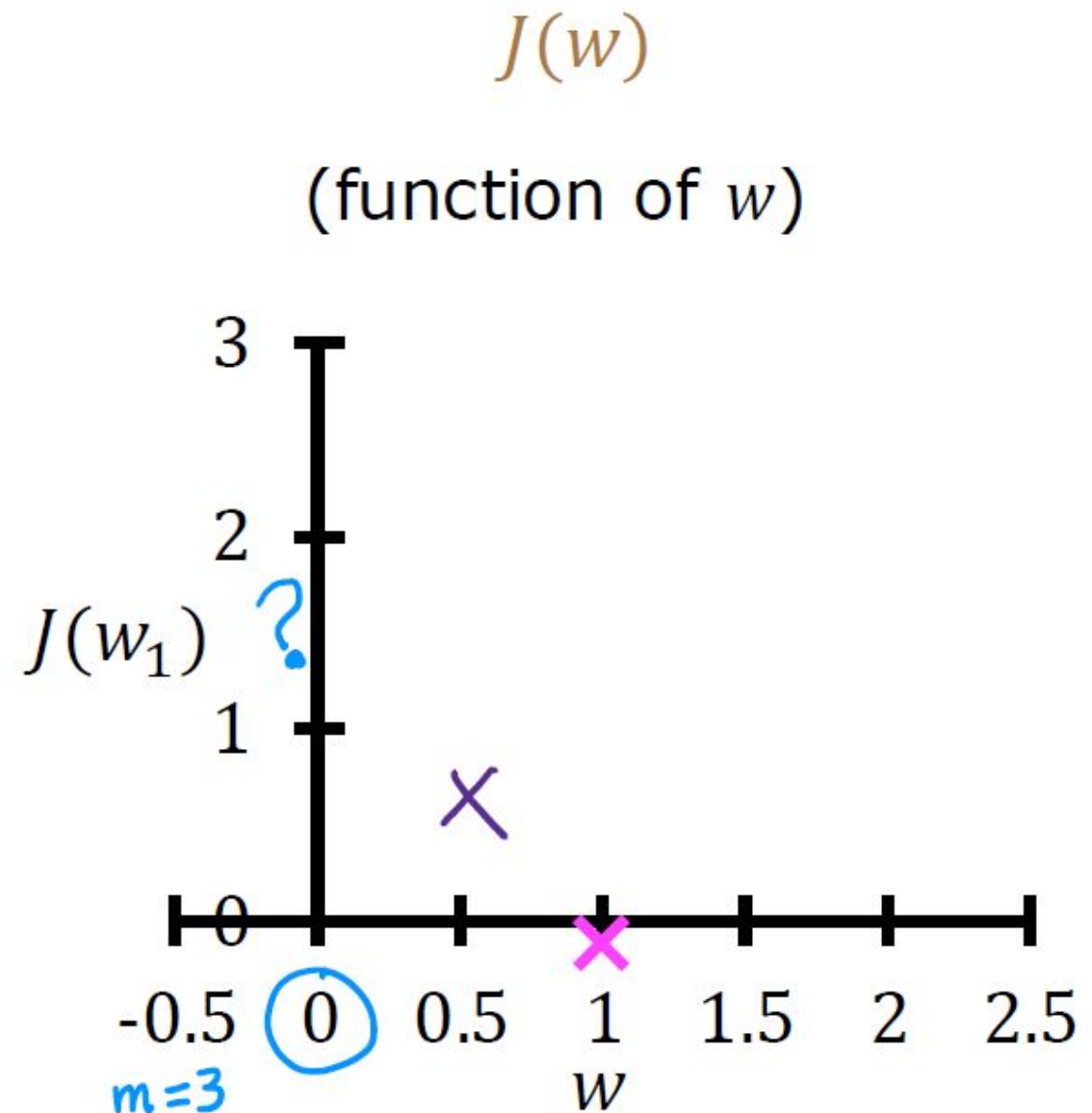
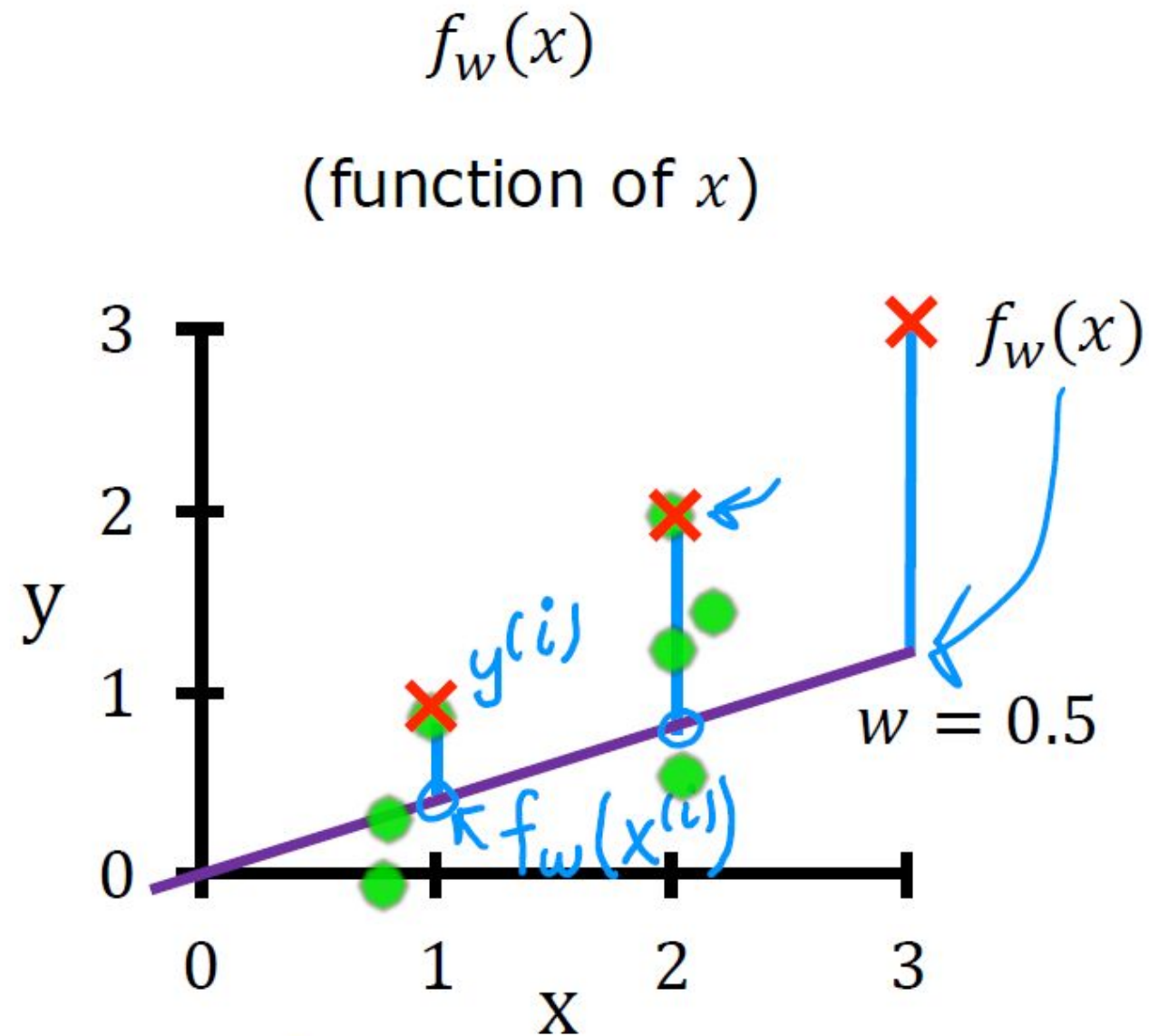
$$\underline{J(w)} = \frac{1}{2m} \sum_{i=1}^m (\underline{f_w(x^{(i)})} - y^{(i)})^2$$

$$\underset{\underline{w}}{\text{minimize}} \underline{J(w)}$$

2. Cost function

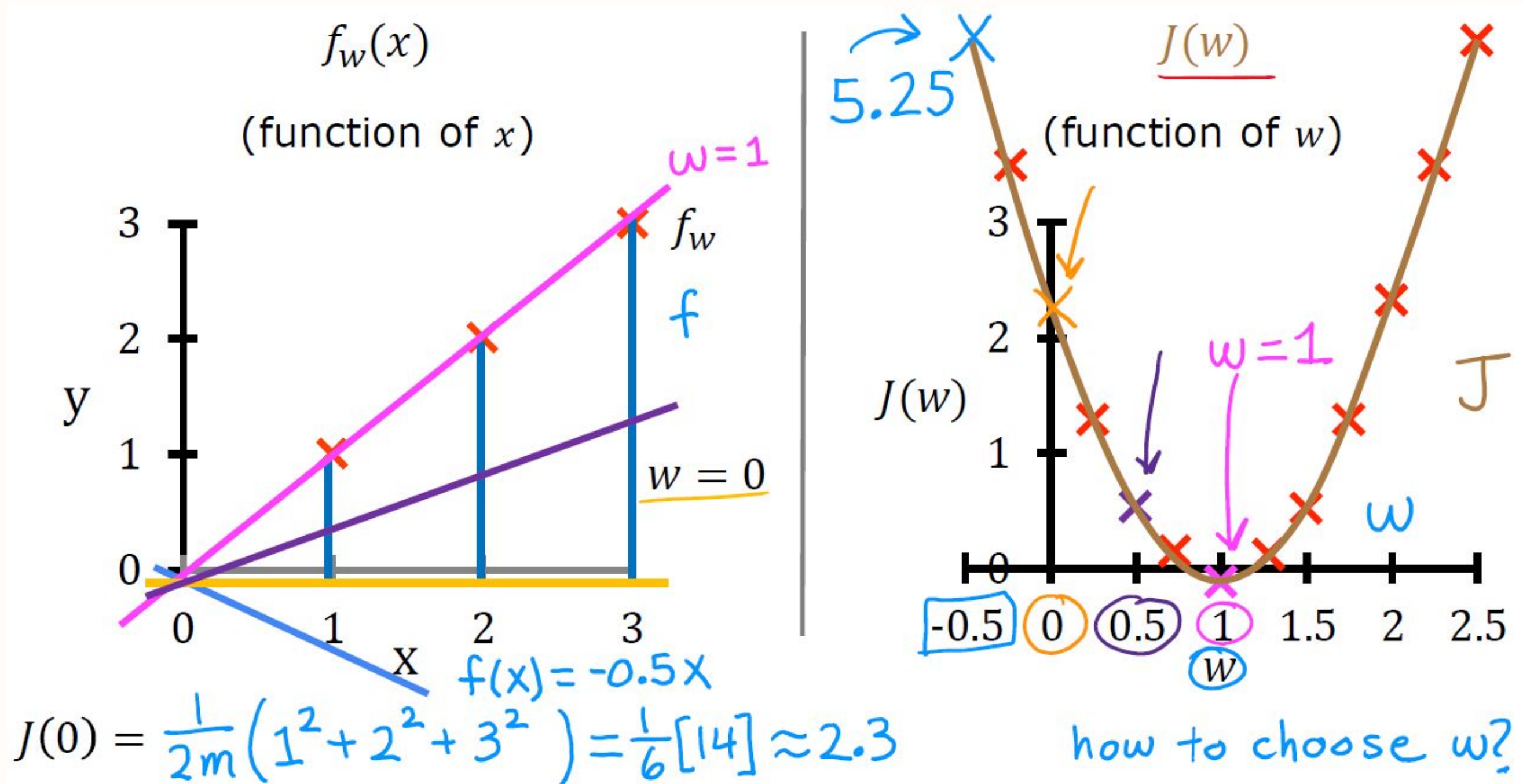


2. Cost function



$$J(0.5) = \frac{1}{2m} [(0.5-1)^2 + (1-2)^2 + (1.5-3)^2] = \frac{1}{2 \times 3} [3.5] = \frac{3.5}{6} \approx 0.58$$

2. Cost function



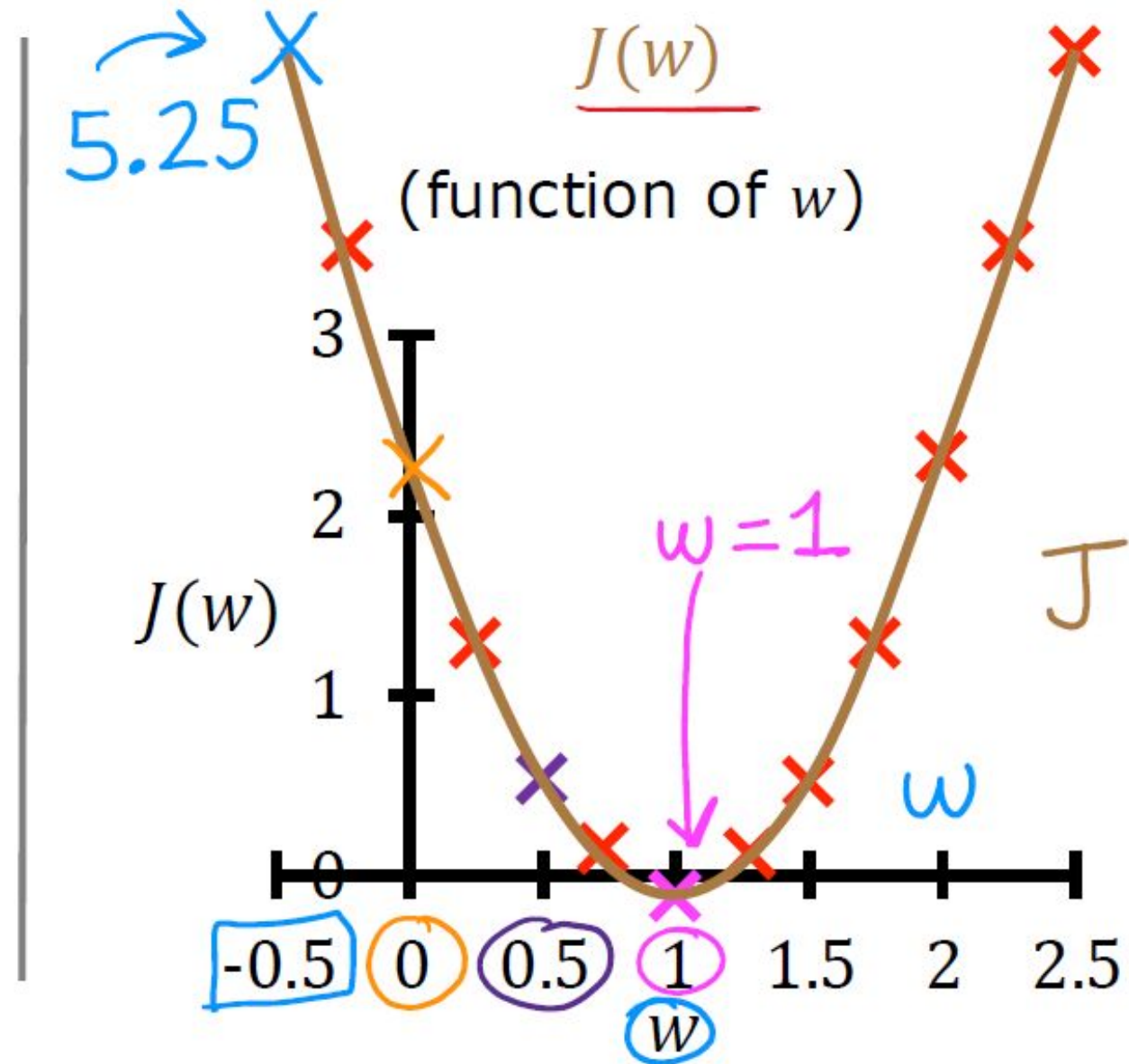
2. Cost function

goal of linear regression:

$$\underset{w}{\text{minimize}} J(w)$$

general case:

$$\underset{w,b}{\text{minimize}} J(w, b)$$



choose w to minimize $J(w)$

3. Visualizing the Cost Function

Model $f_{w,b}(x) = wx + b$

Parameters w, b ~~before: $b=0$~~

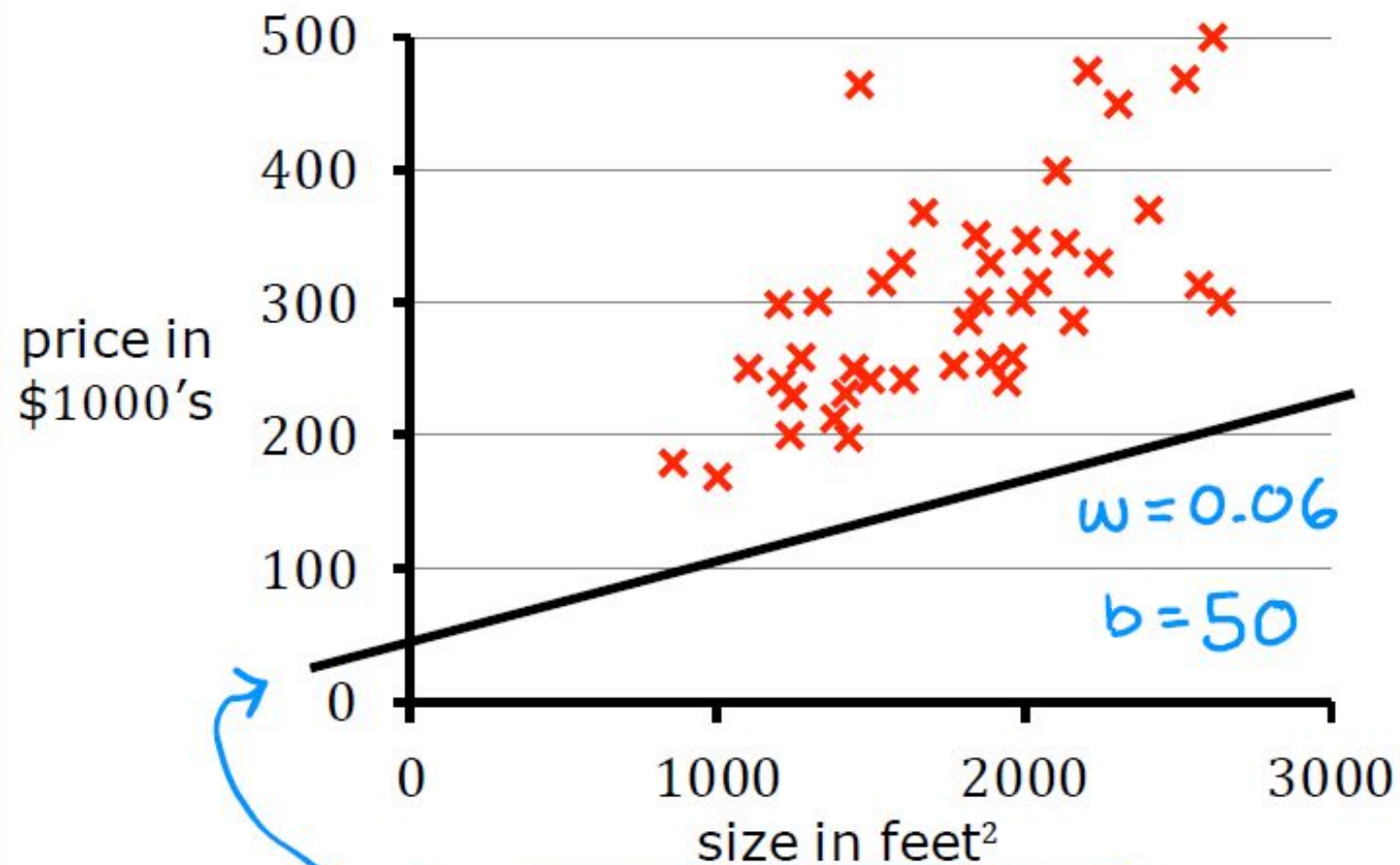
Cost Function $J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$

Objective $\underset{w,b}{\text{minimize}} J(w, b)$

3. Visualizing the Cost Function

$f_{w,b}$

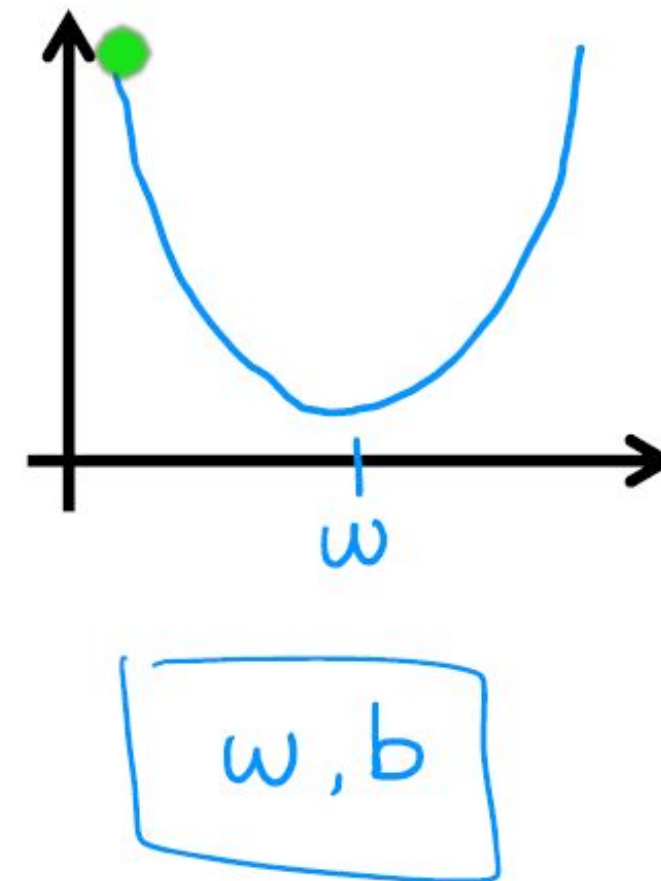
(function of x)



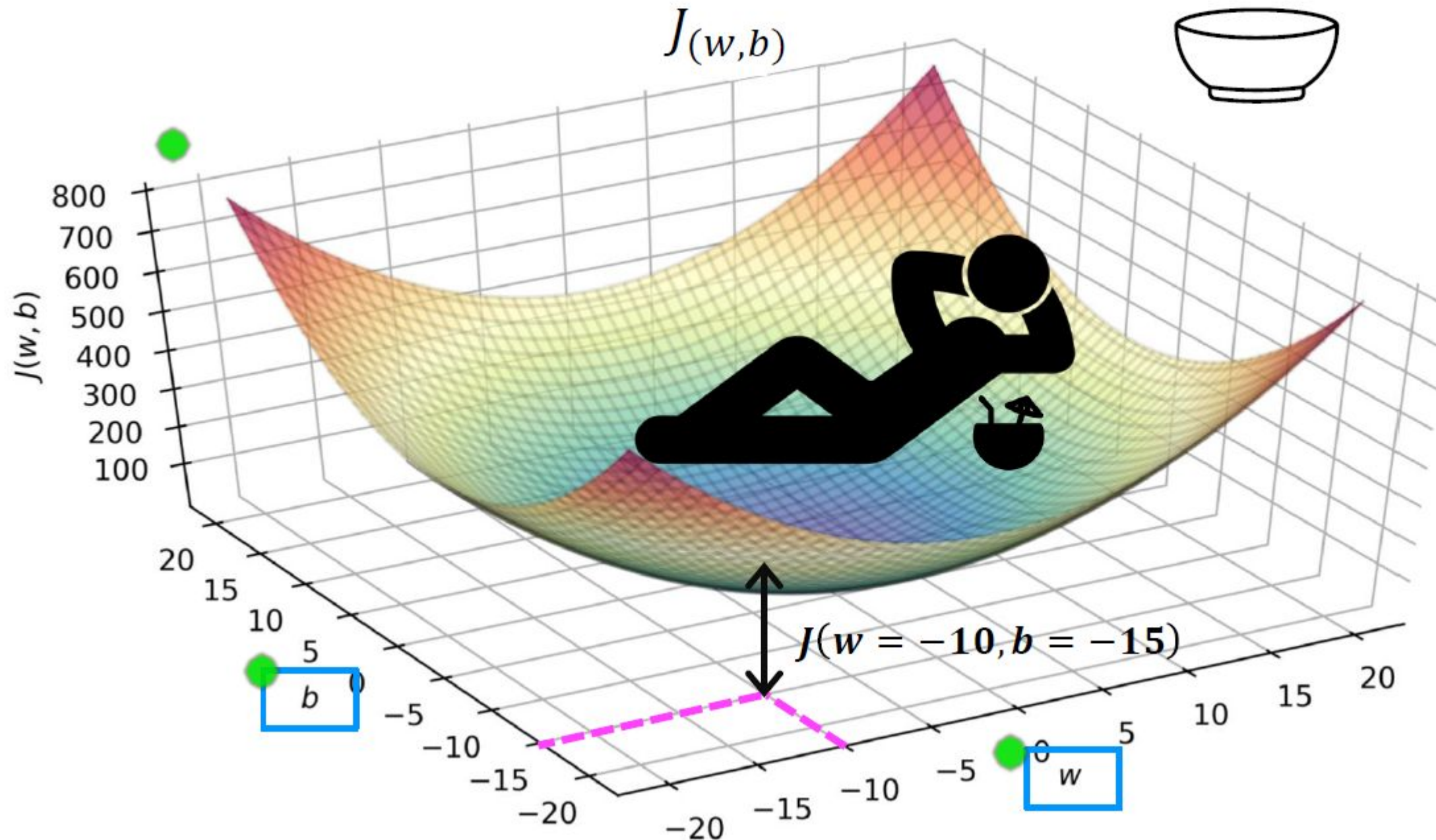
$$f_{w,b}(x) = 0.06x + 50$$

J

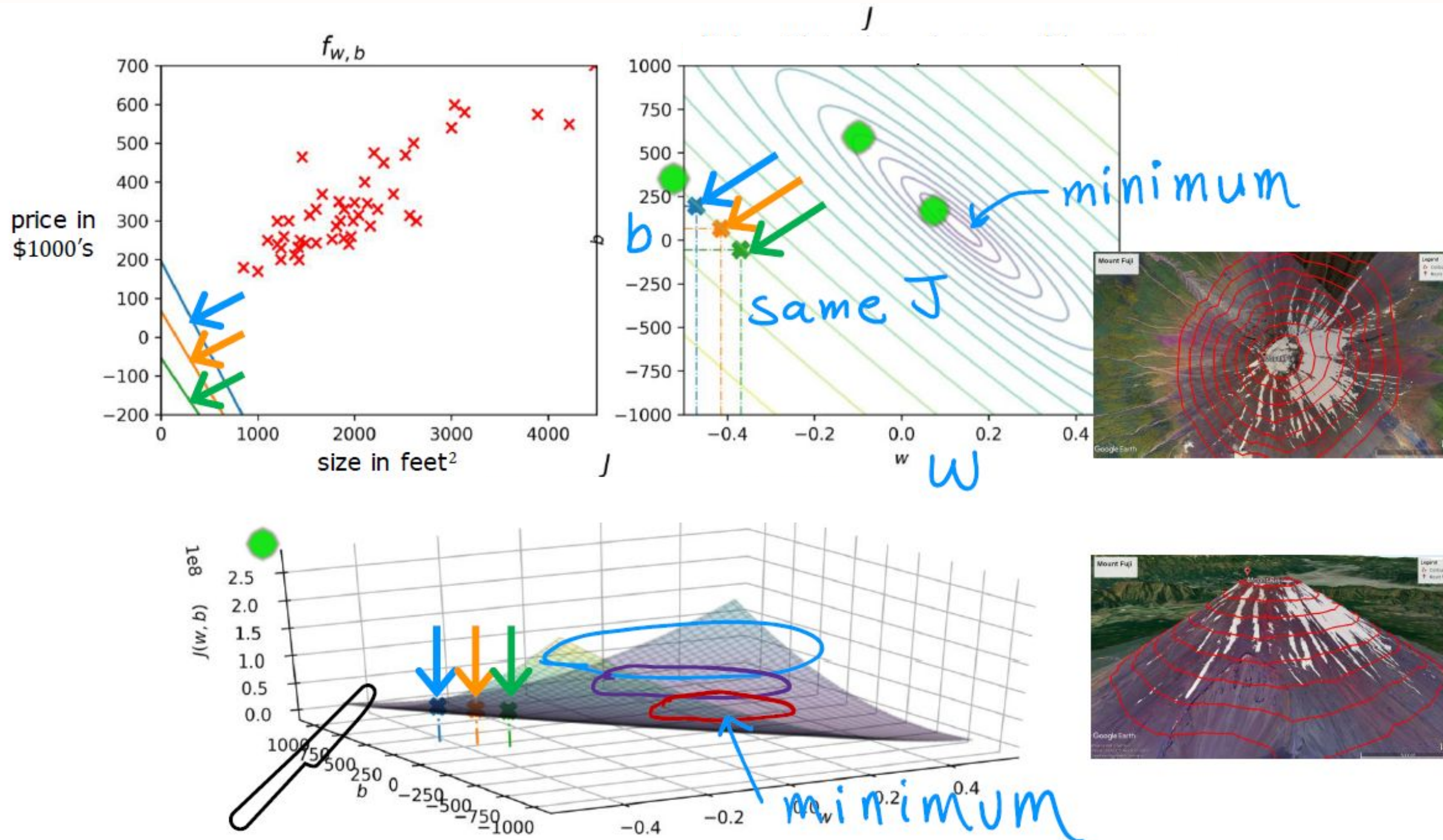
(function of w, b)



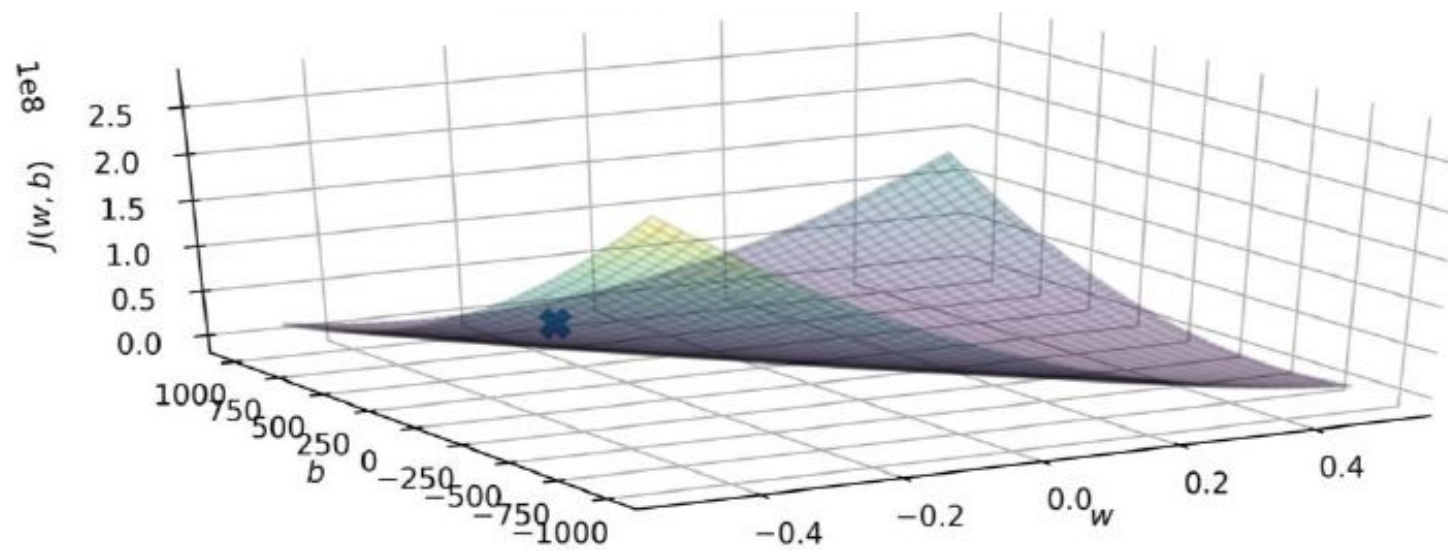
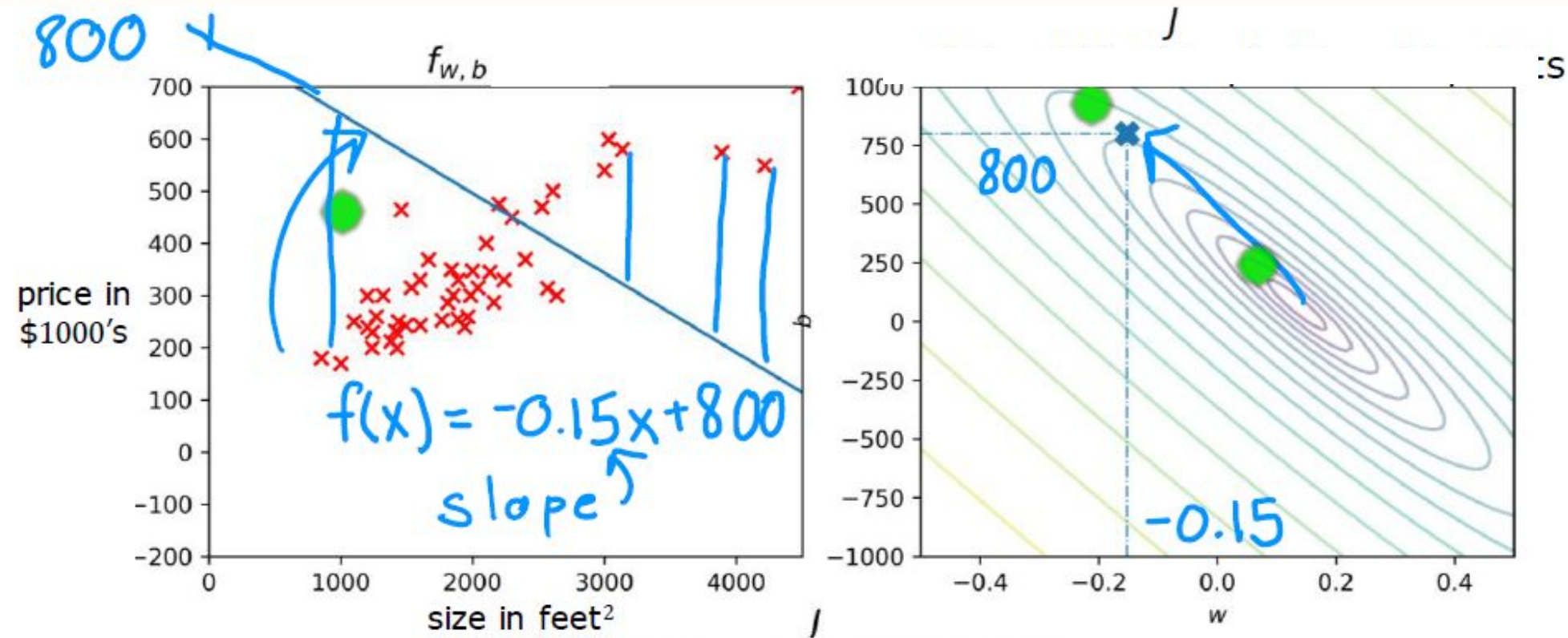
3. Visualizing the Cost Function



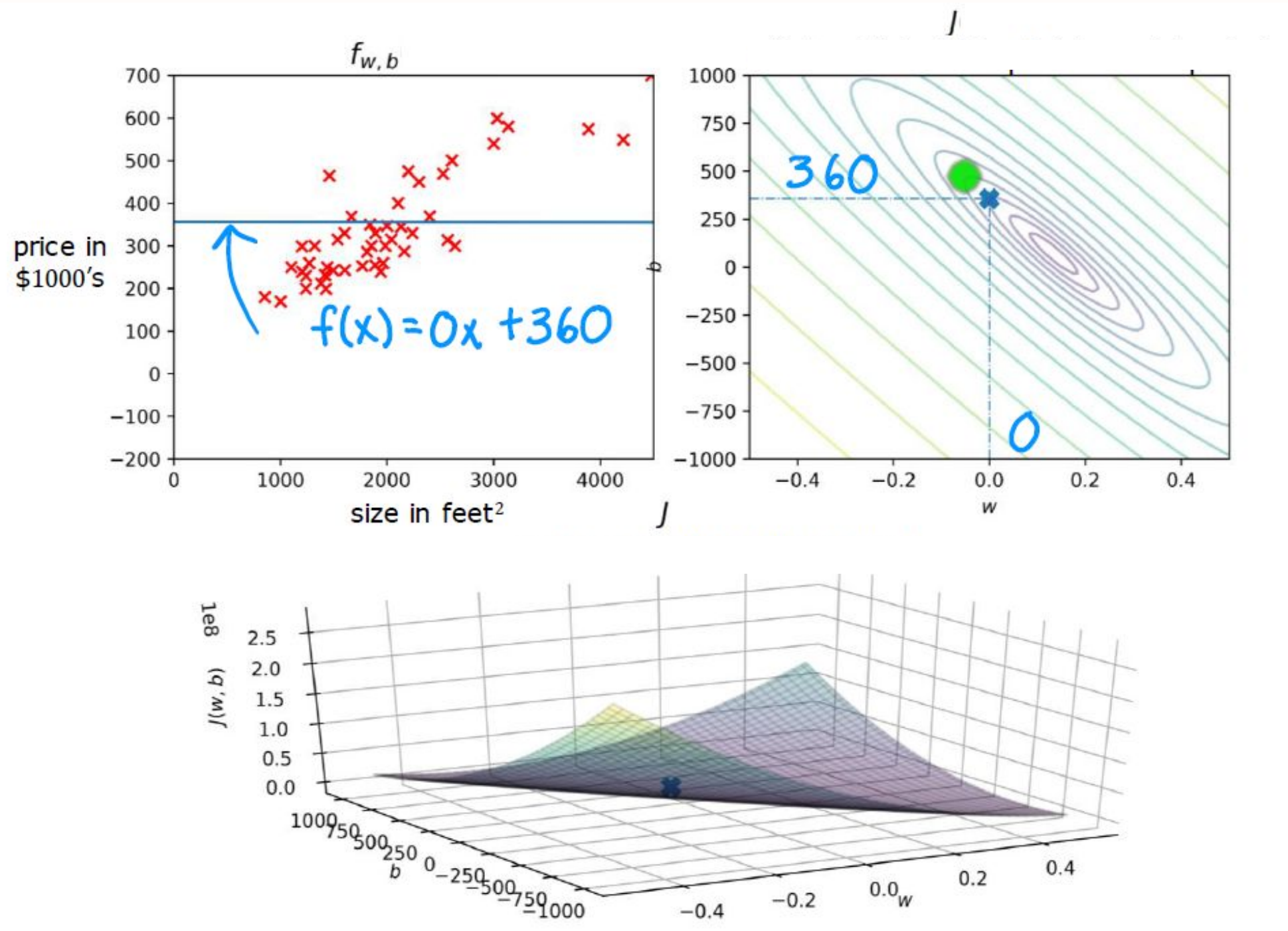
3. Visualizing the Cost Function



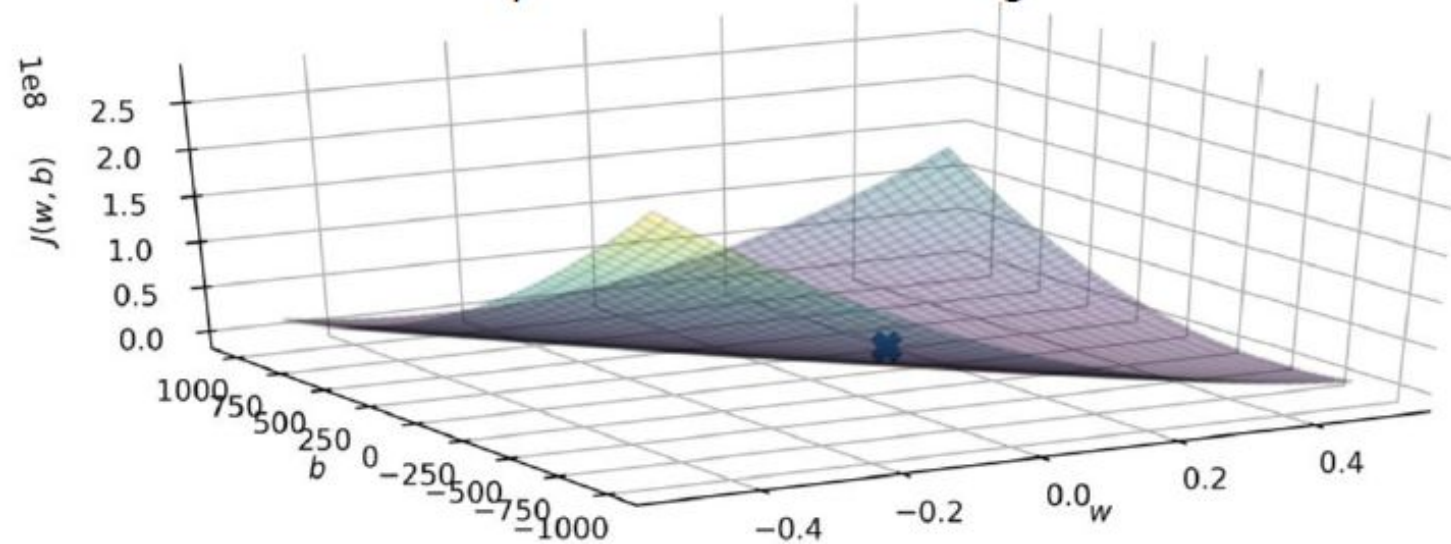
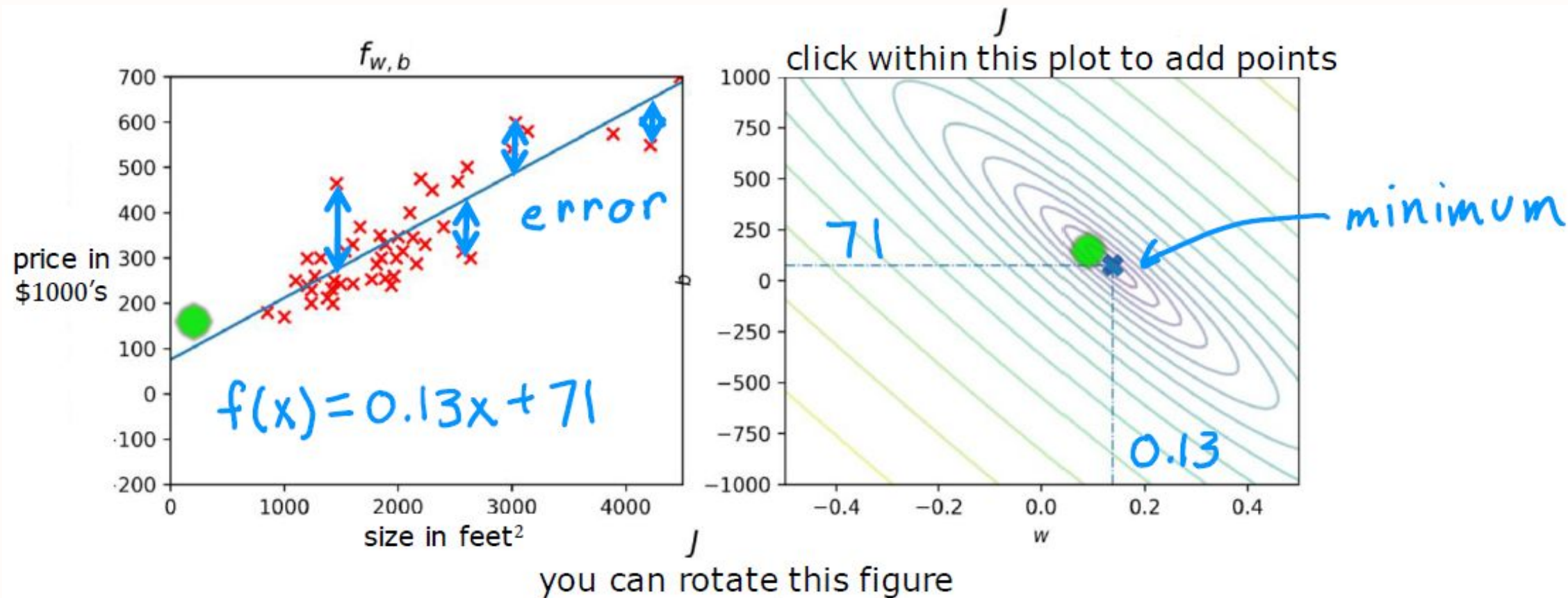
4. Visualization examples



4. Visualization examples



4. Visualization examples



5. Gradient Descent

Have some function $J(w, b)$ *for linear regression
or any function*

Want $\min_{w, b} J(w, b)$ $\min_{w_1, \dots, w_n, b} \underline{J(w_1, w_2, \dots, w_n, b)}$

Outline:

Start with some w, b *(set $w=0, b=0$)*

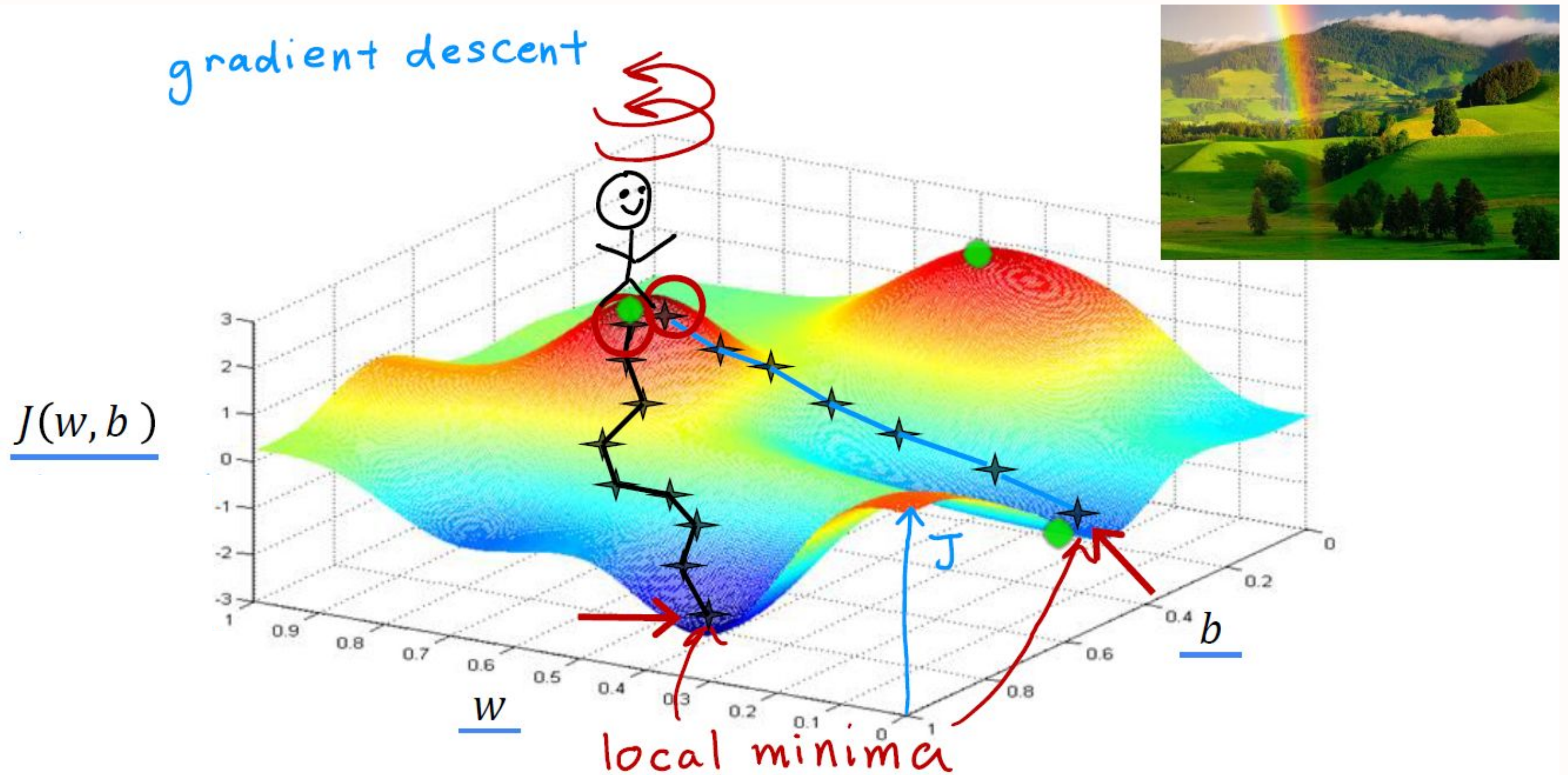
Keep changing w, b to reduce $J(w, b)$

Until we settle at or near a minimum

may have >1 minimum



5. Gradient Descent



6. Implementing Gradient Descent

Repeat until convergence

$$\left\{ \begin{array}{l} \underline{w} = w - \alpha \frac{\partial}{\partial w} J(w, b) \\ \underline{b} = b - \alpha \frac{\partial}{\partial b} J(w, b) \end{array} \right.$$

Learning rate
Derivative

Simultaneously
update w and b

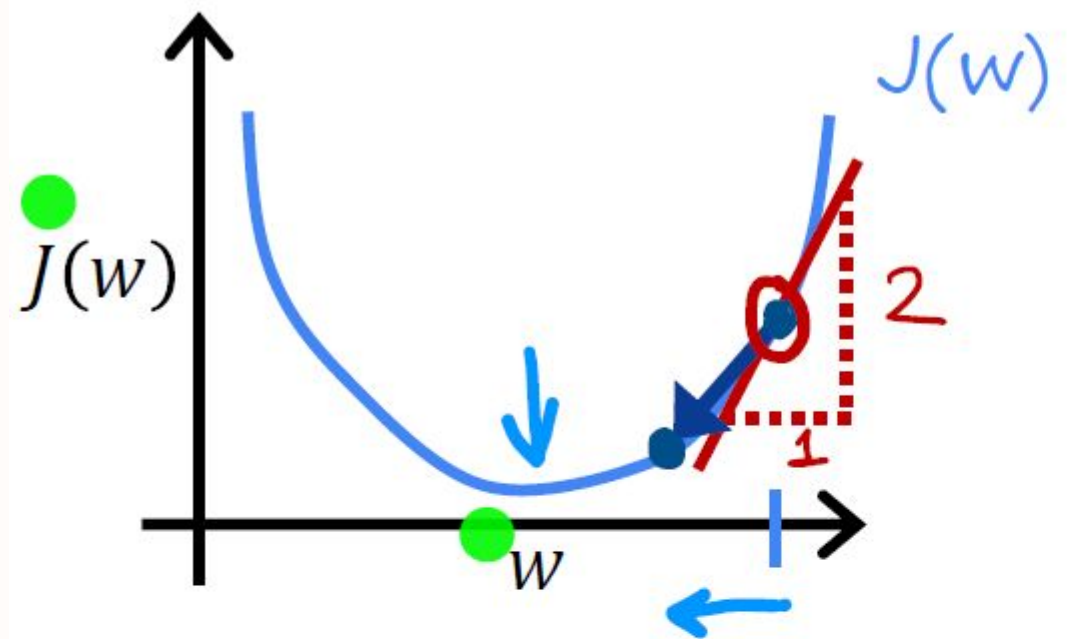
Correct: Simultaneous update

$$\begin{aligned} tmp_w &= w - \alpha \frac{\partial}{\partial w} J(w, b) \\ tmp_b &= b - \alpha \frac{\partial}{\partial b} J(w, b) \\ w &= tmp_w \\ b &= tmp_b \end{aligned}$$

Incorrect

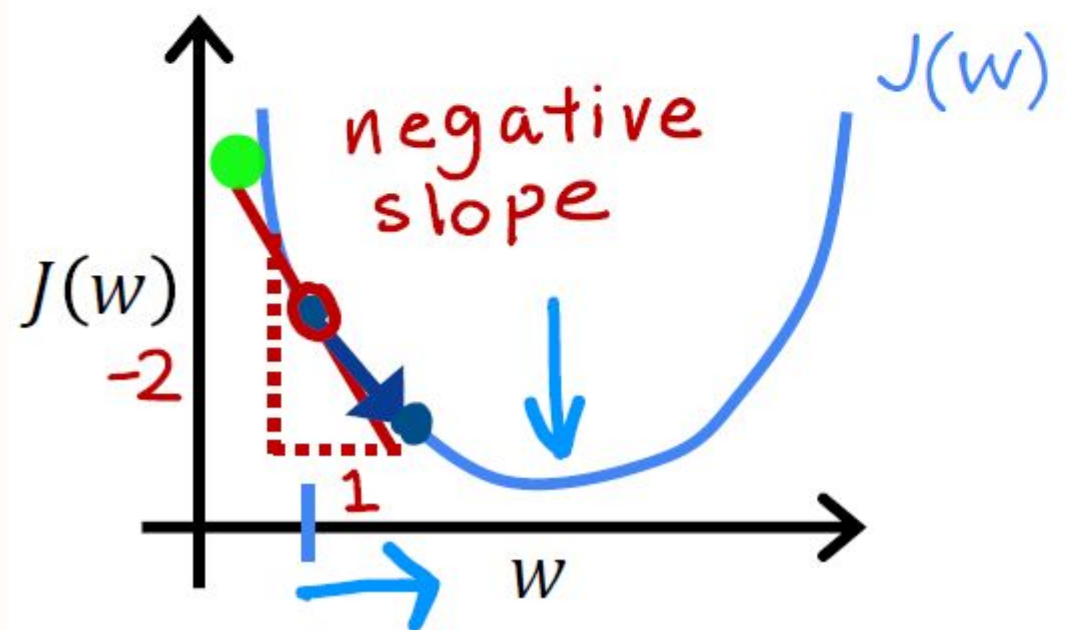
$$\begin{aligned} tmp_w &= w - \alpha \frac{\partial}{\partial w} J(w, b) \\ \underline{w} &= tmp_w \\ tmp_b &= b - \alpha \frac{\partial}{\partial b} J(\underline{w}, b) \\ \underline{b} &= tmp_b \end{aligned}$$

6. Implementing Gradient Descent



$$w = w - \alpha \underbrace{\frac{d}{dw} J(w)}_{>0}$$

$$w = w - \underline{\alpha} \cdot (\text{positive number})$$



$$\underbrace{\frac{d}{dw} J(w)}_{<0}$$

$$w = \underbrace{w}_{\uparrow} - \alpha \cdot \underbrace{(\text{negative number})}_{\uparrow}$$

7. Learning Rate

$$w = w - \alpha \frac{d}{dw} J(w)$$

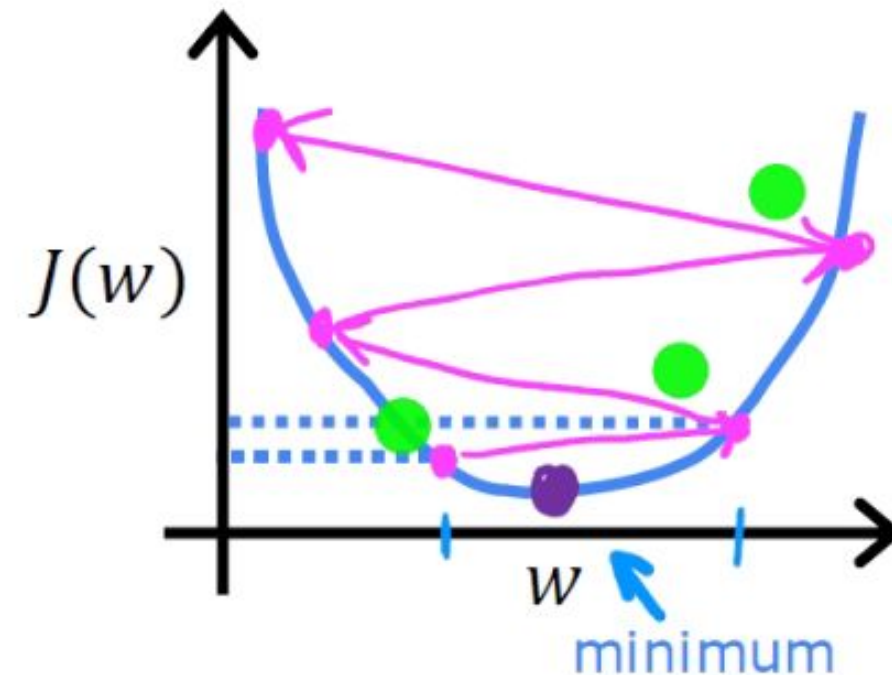
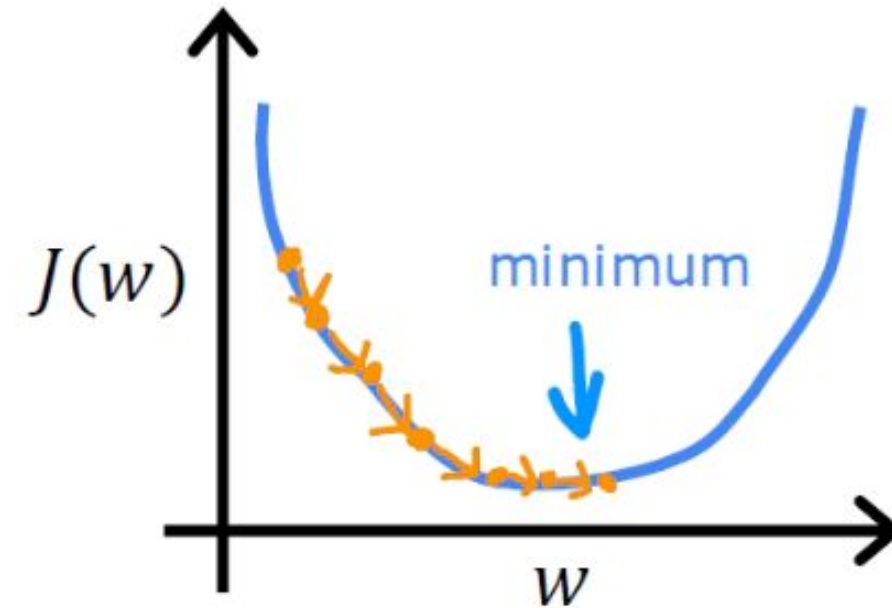
If α is too small...

Gradient descent may be slow.

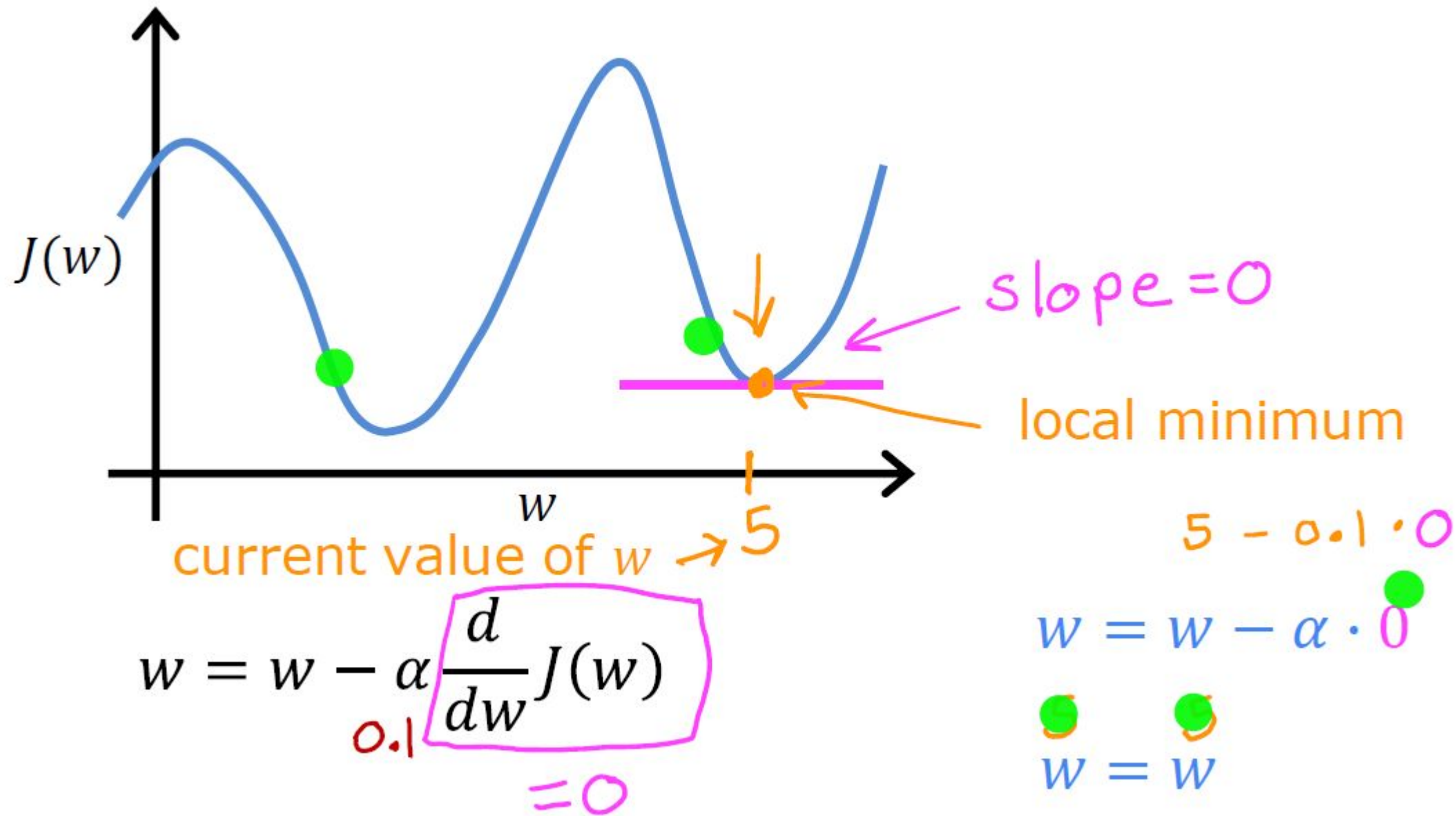
If α is too large...

Gradient descent may:

- Overshoot, never reach minimum
- Fail to converge, diverge



7. Learning Rate



8. Gradient Descent for Linear Regression

Linear regression model

$$f_{w,b}(x) = wx + b$$

Cost function

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Gradient descent algorithm

repeat until convergence {

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b) \rightarrow \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b) \rightarrow \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

}

8. Gradient Descent for Linear Regression

$$\frac{\partial}{\partial w} J(w, b) = \frac{d}{dw} \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2 = \frac{d}{dw} \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)})^2$$

$$= \cancel{\frac{1}{2m}} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)}) \cancel{2} x^{(i)} = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$\frac{\partial}{\partial b} J(w, b) = \frac{d}{db} \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2 = \frac{d}{db} \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)})^2$$

$$= \cancel{\frac{1}{2m}} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)}) \cancel{2} = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

no $x^{(i)}$

8. Gradient Descent for Linear Regression

Gradient descent algorithm

repeat until convergence {

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$
$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

}

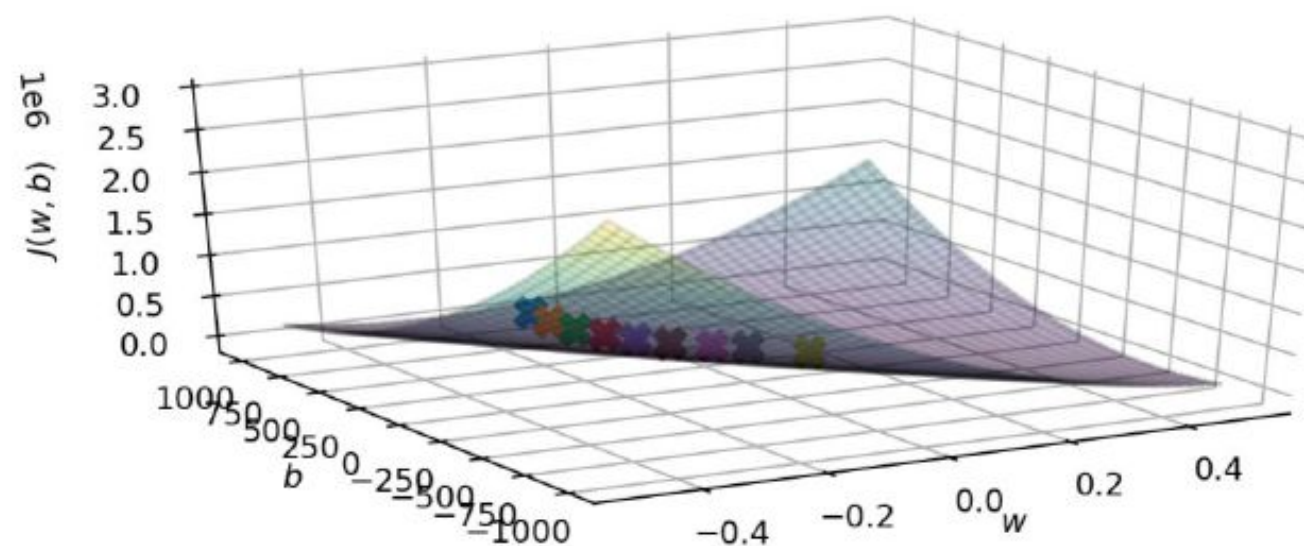
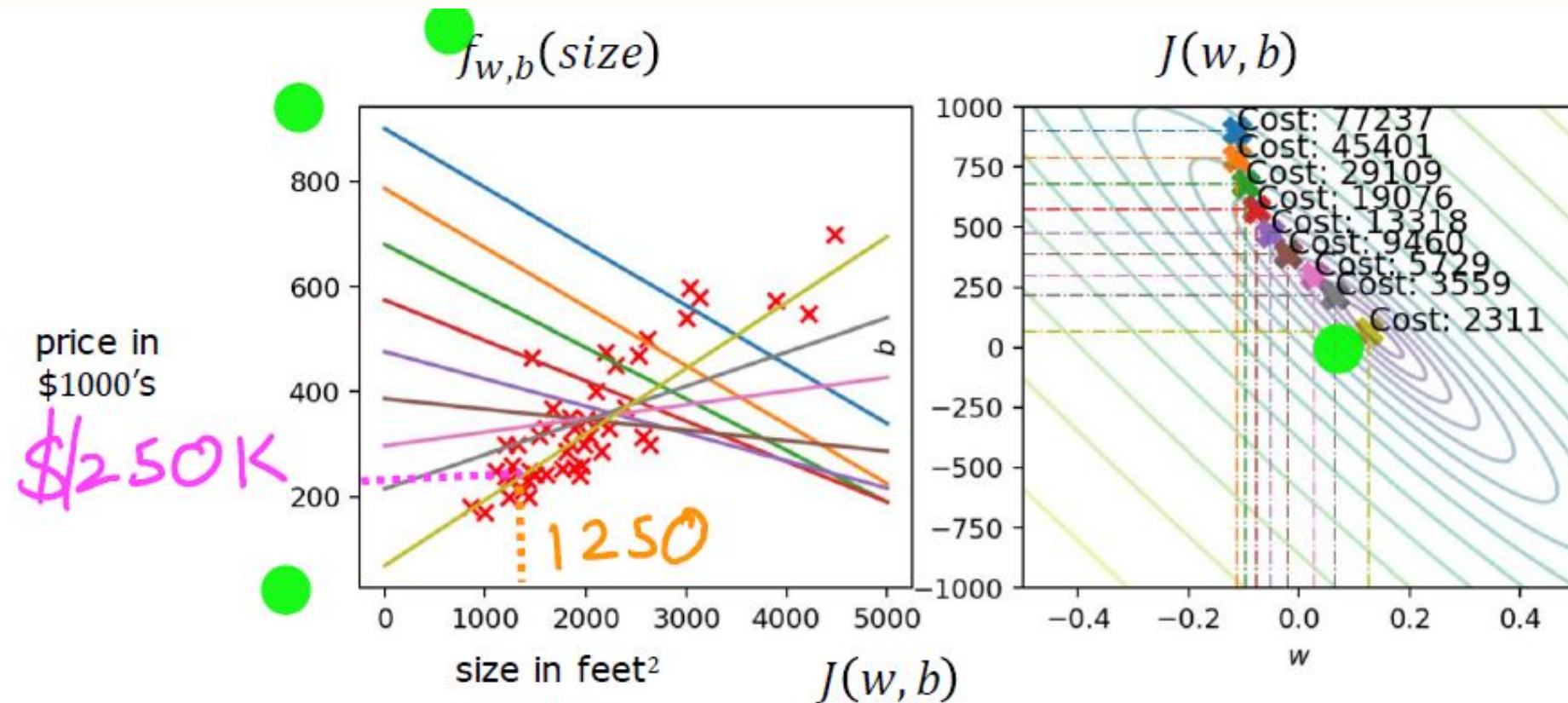
$\frac{d}{dw} J(w, b)$

$\frac{d}{db} J(w, b)$

Update w and b simultaneously

$f_{w,b}(x^{(i)}) = wx^{(i)} + b$

8. Gradient Descent for Linear Regression



Linear Regression with Multiple Variable

1. Multiple Features (variables)

	Size in feet ² x_1	Number of bedrooms x_2	Number of floors x_3	Age of home in years x_4	Price (\$) in \$1000's
	2104	5	1	45	460
$i=2$	1416	3	2	40	232
	1534	3	2	30	315
	852	2	1	36	178
	...	...	...	...	...

$x_j = j^{th}$ feature

n = number of features

$\vec{x}^{(i)}$ = features of i^{th} training example

$x_j^{(i)}$ = value of feature j in i^{th} training example

$j=1...4$
 $n=4$

$\vec{x}^{(2)} = [1416 \ 3 \ 2 \ 40]$
 $x_3^{(2)} = 2$

2. Model

Previously: $f_{w,b}(x) = wx + b$

example $f_{w,b}(x) = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$

$$f_{w,b}(x) = \underset{\substack{\uparrow \\ \text{size}}}{0.1}x_1 + \underset{\substack{\uparrow \\ \text{\# bedrooms}}}{4}x_2 + \underset{\substack{\uparrow \\ \text{\# floors}}}{10}x_3 + \underset{\substack{\uparrow \\ \text{years}}}{-2}x_4 + \underset{\substack{\uparrow \\ \text{base price}}}{80}$$

$$f_{w,b}(x) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

2. Model

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

$\vec{w} = [w_1 \ w_2 \ w_3 \ \dots \ w_n]$ parameters of the model
 b is a number

vector $\vec{x} = [x_1 \ x_2 \ x_3 \ \dots \ x_n]$

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n + b$$

dot product

3. Vectorization

Parameters and features

$$\vec{w} = [w_1 \ w_2 \ w_3] \quad n=3$$

b is a number

$$\vec{x} = [x_1 \ x_2 \ x_3]$$

linear algebra: count from 1

NumPy 

$w[0] \ w[1] \ w[2]$

```
w = np.array([1.0, 2.5, -3.3])
```

```
b = 4  $x[0] \ x[1] \ x[2]$ 
```

```
x = np.array([10, 20, 30])
```

code: count from 0

Without vectorization $n=100,000$

$$f_{\vec{w},b}(\vec{x}) = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

```
f = w[0] * x[0] +  
     w[1] * x[1] +  
     w[2] * x[2] + b
```



Without vectorization

$$f_{\vec{w},b}(\vec{x}) = \left(\sum_{j=1}^n w_j x_j \right) + b \quad \sum_{j=1}^n \rightarrow j=1 \dots n \quad 1, 2, 3$$

$\text{range}(0, n) \rightarrow j=0 \dots n-1$

```
f = 0  $\text{range}(n)$   
for j in range(0, n):  
    f = f + w[j] * x[j]  
f = f + b
```



Vectorization

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

```
f = np.dot(w, x) + b
```



3. Vectorization

Without vectorization

```
for j in range(0,16):  
    f = f + w[j] * x[j]
```

t_0

$$f + w[0] * x[0]$$

t_1

$$f + w[1] * x[1]$$

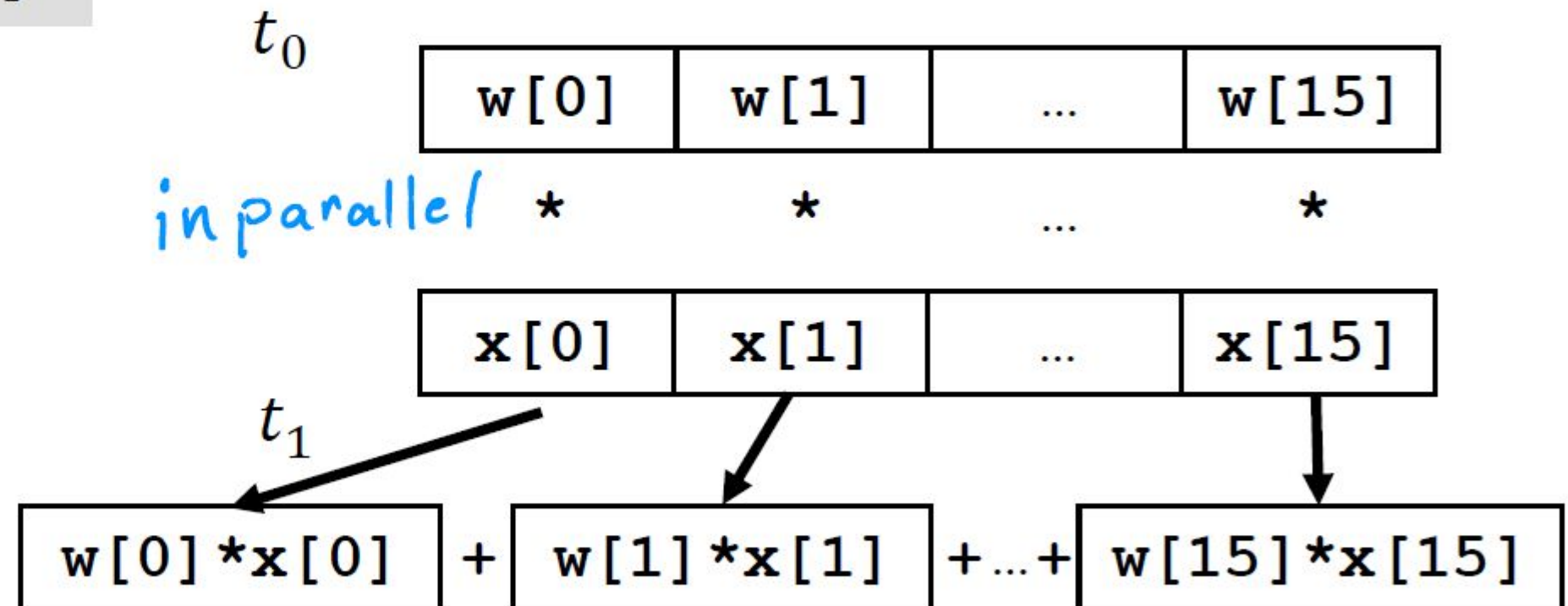
...

t_{15}

$$f + w[15] * x[15]$$

Vectorization

```
np.dot(w,x)
```



efficient → scale to large datasets

3. Vectorization

Gradient descent $\vec{w} = (w_1 \ w_2 \ \dots \ w_{16})$ ~~b~~ parameters
derivatives $\vec{d} = (d_1 \ d_2 \ \dots \ d_{16})$

```
w = np.array([0.5, 1.3, ... 3.4])
```

```
d = np.array([0.3, 0.2, ... 0.4])
```

compute $w_j = w_j - \underbrace{0.1}_{\text{learning rate } \alpha} d_j$ for $j = 1 \dots 16$

Without vectorization

$$w_1 = w_1 - 0.1d_1$$

$$w_2 = w_2 - 0.1d_2$$

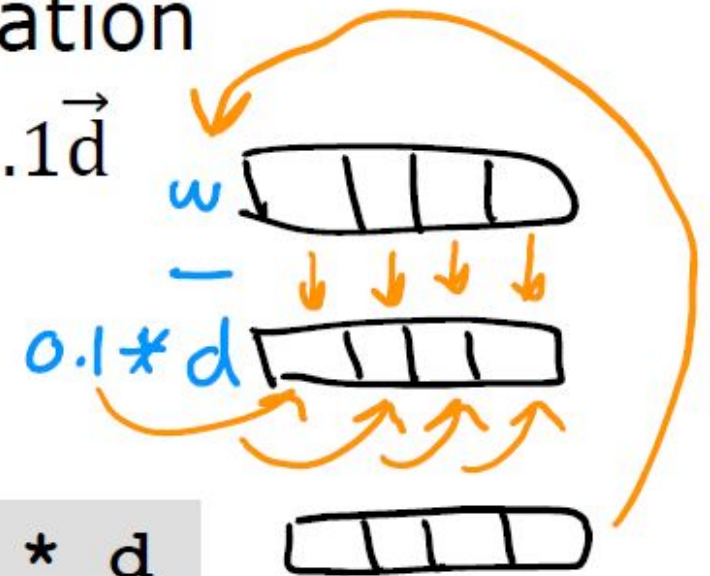
$$\vdots$$

$$w_{16} = w_{16} - 0.1d_{16}$$

```
for j in range(0,16):  
    w[j] = w[j] - 0.1 * d[j]
```

With vectorization

$$\vec{w} = \vec{w} - 0.1\vec{d}$$



```
w = w - 0.1 * d
```


4. Gradient Descent for Multiple Regression

Previous notation

Parameters

$$w_1, \dots, w_n$$

● b

Model

$$f_{\vec{w},b}(\vec{x}) = w_1 x_1 + \dots + w_n x_n + b$$

Cost function

● $J(\underbrace{w_1, \dots, w_n}_\text{vector}, b)$

Gradient descent

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\underbrace{w_1, \dots, w_n}_\text{vector}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\underbrace{w_1, \dots, w_n}_\text{vector}, b)$$

}

Vector notation

↙ vector of length n
 $\vec{w} = [w_1 \ \dots \ w_n]$

b still a number

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

↗ dot product
 $J(\vec{w}, b)$

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

}

4. Gradient Descent for Multiple Regression

One feature

repeat {

$$\underline{w} = w - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) \underline{x^{(i)}})$$

$\frac{\partial}{\partial w} J(w, b)$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

simultaneously update w, b

}

n features ($n \geq 2$)

repeat {

$$\begin{aligned} j=1 \quad \underline{w_1} &= w_1 - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \underline{x_1^{(i)}} \\ &\vdots \end{aligned}$$

$\frac{\partial}{\partial w_1} J(\vec{w}, b)$

$j=n$

$$w_n = w_n - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_n^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

simultaneously update

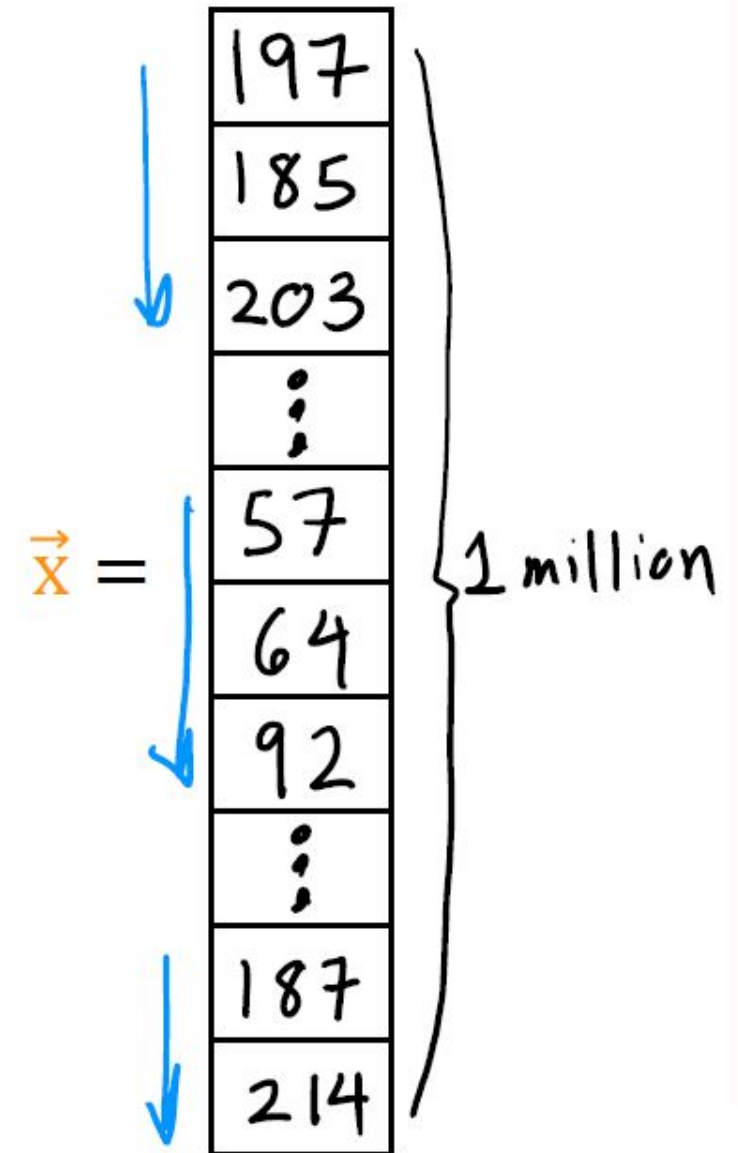
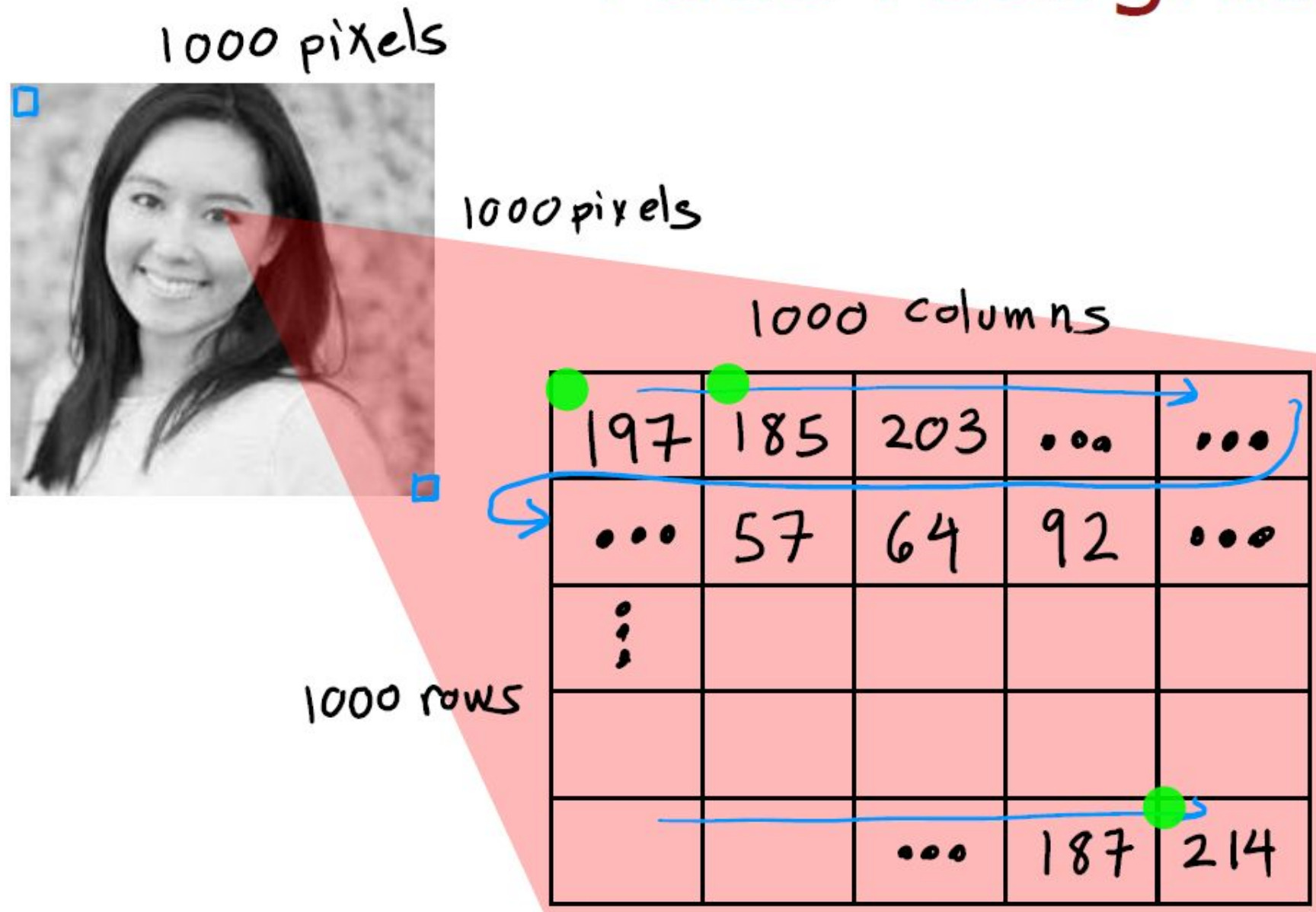
w_j (for $j = 1, \dots, n$) and b

}

Neural Network

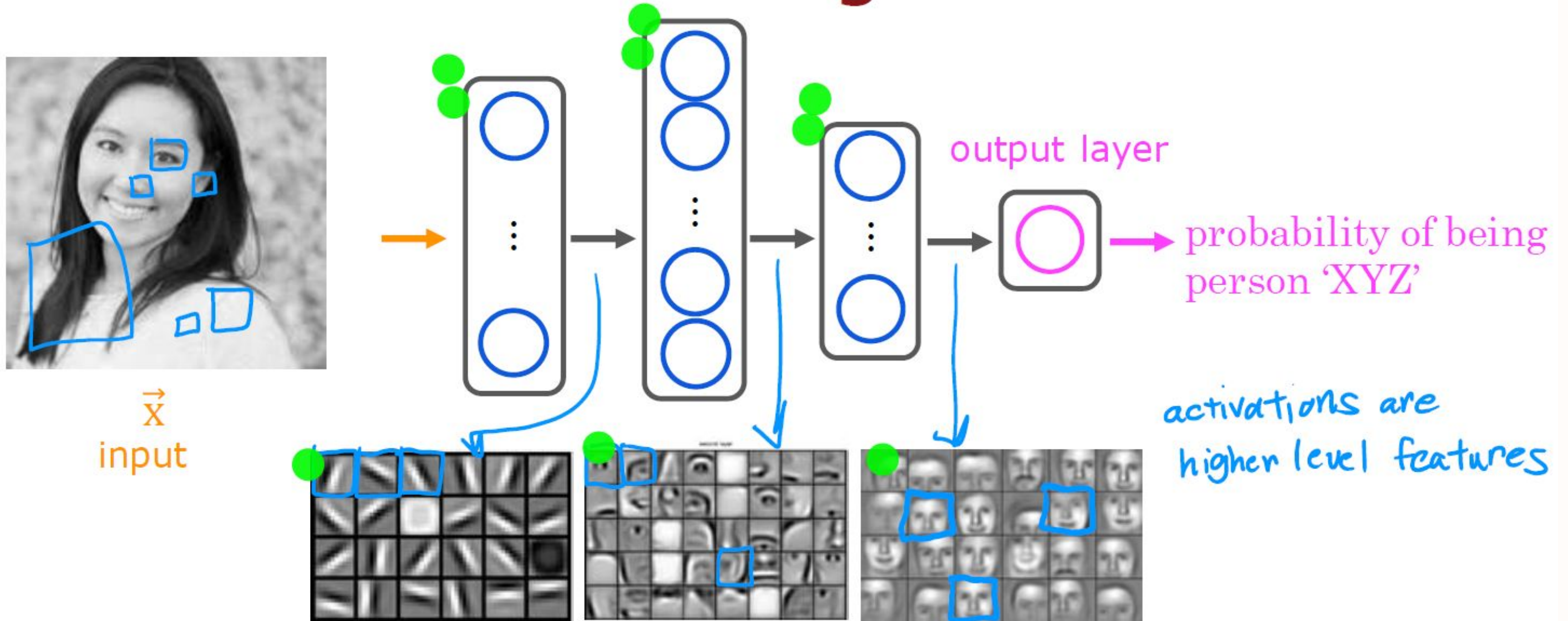
1. Example: Recognizing Images

Face recognition



1. Example: Recognizing Images

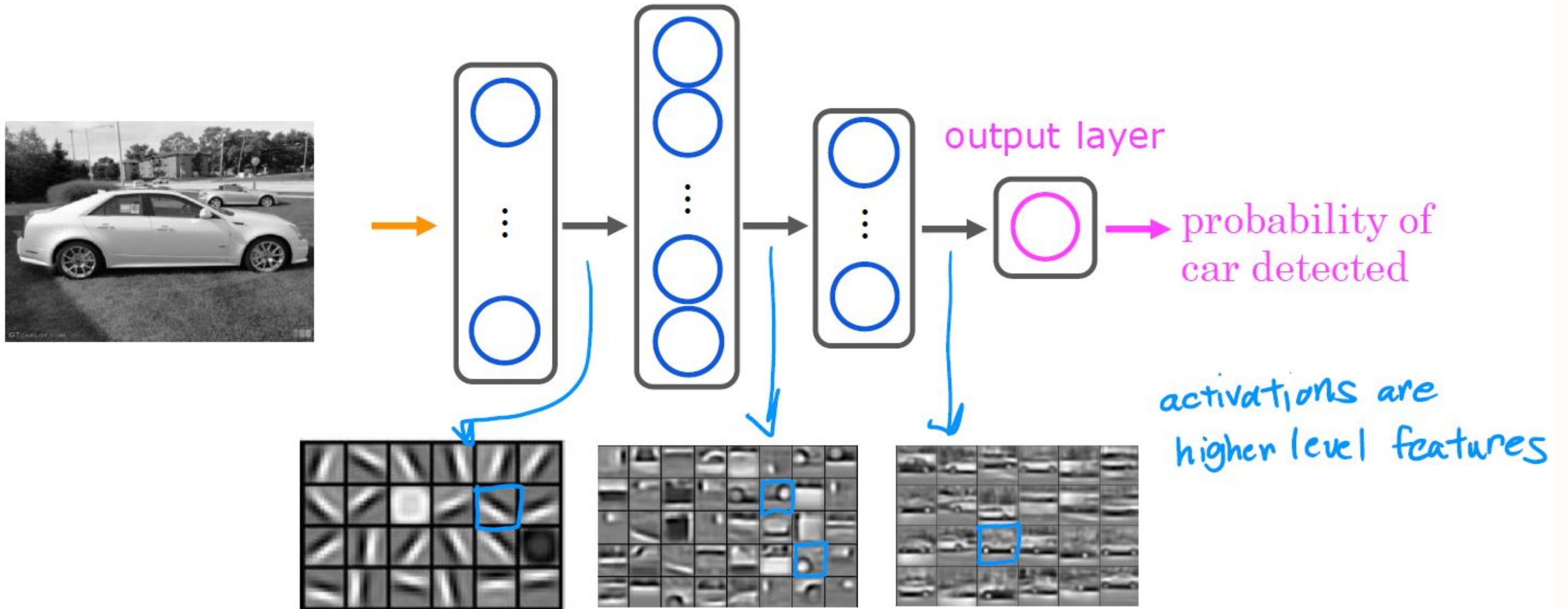
Face recognition



source: Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations
by Honglak Lee, Roger Grosse, Ranganath Andrew Y. Ng

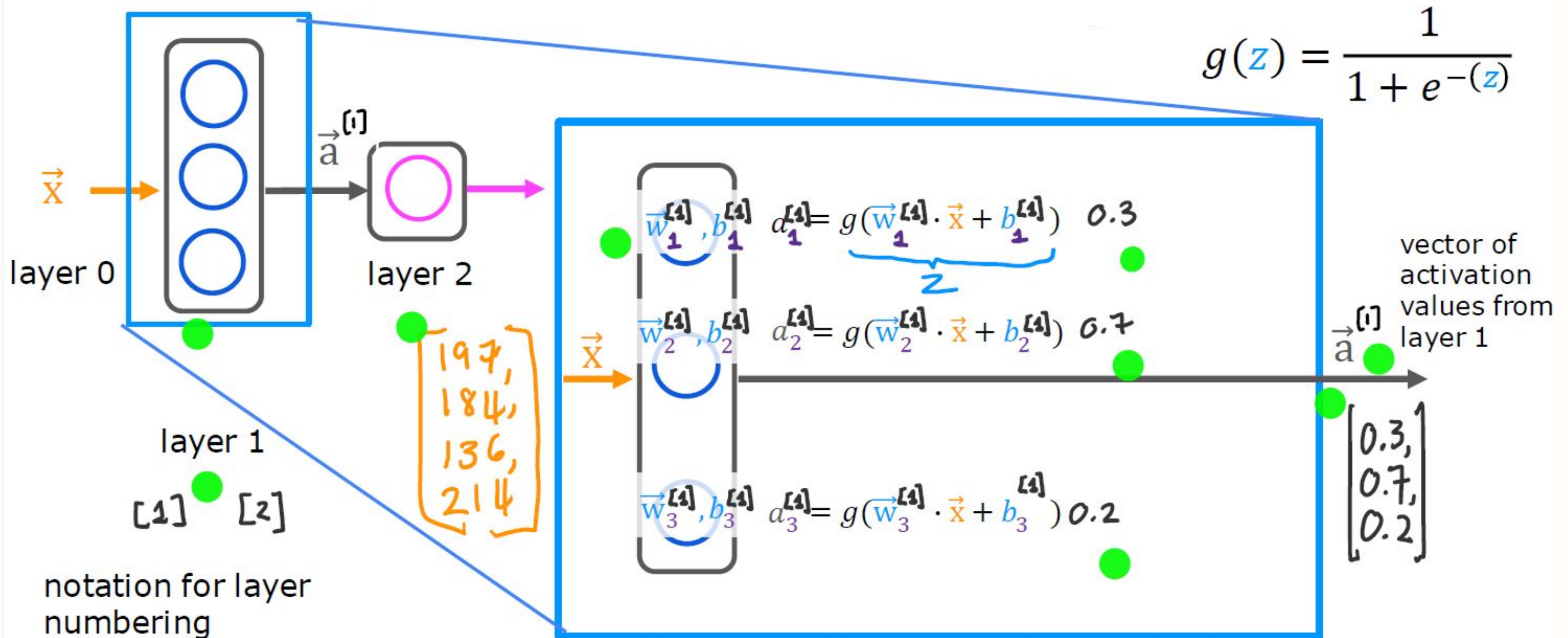
1. Example: Recognizing Images

Car classification

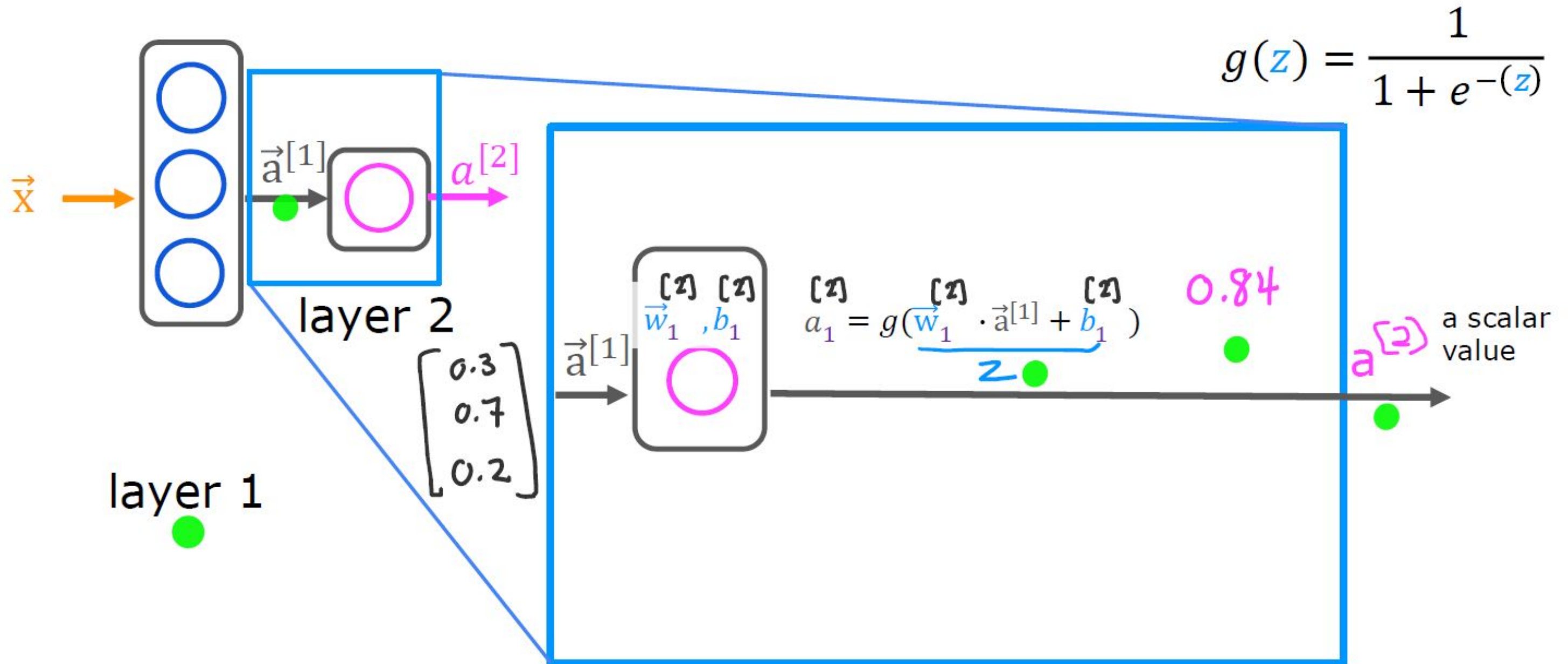


source: Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations
by Honglak Lee, Roger Grosse, Ranganath Andrew Y. Ng

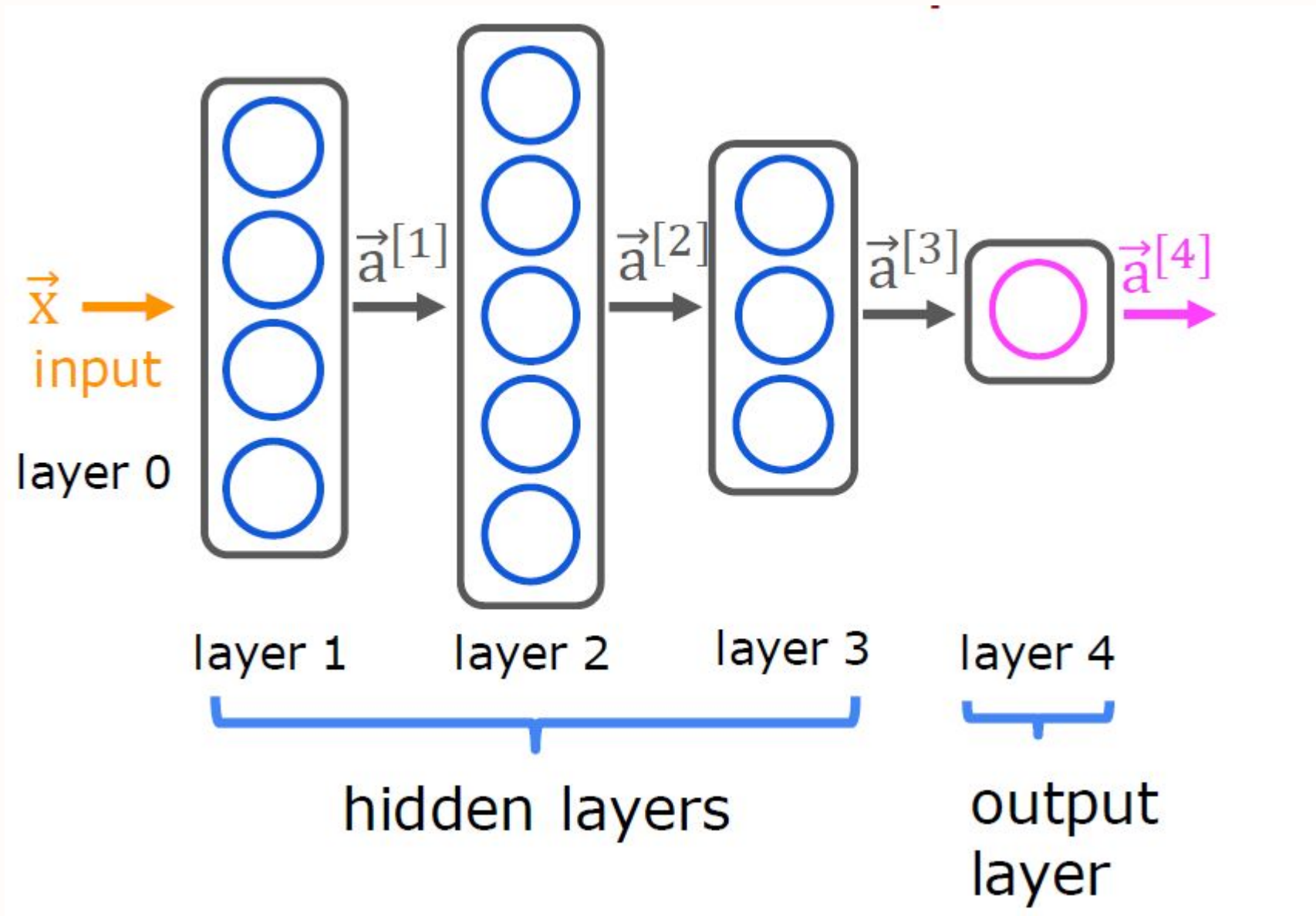
2. Neural network layer



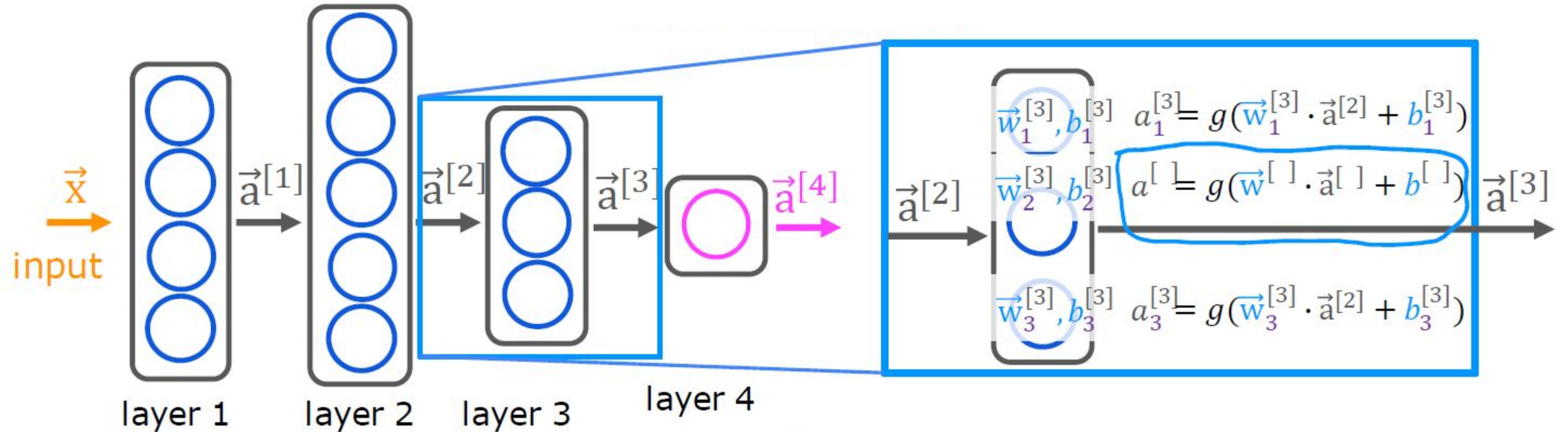
2. Neural network layer



3. More complex neural networks



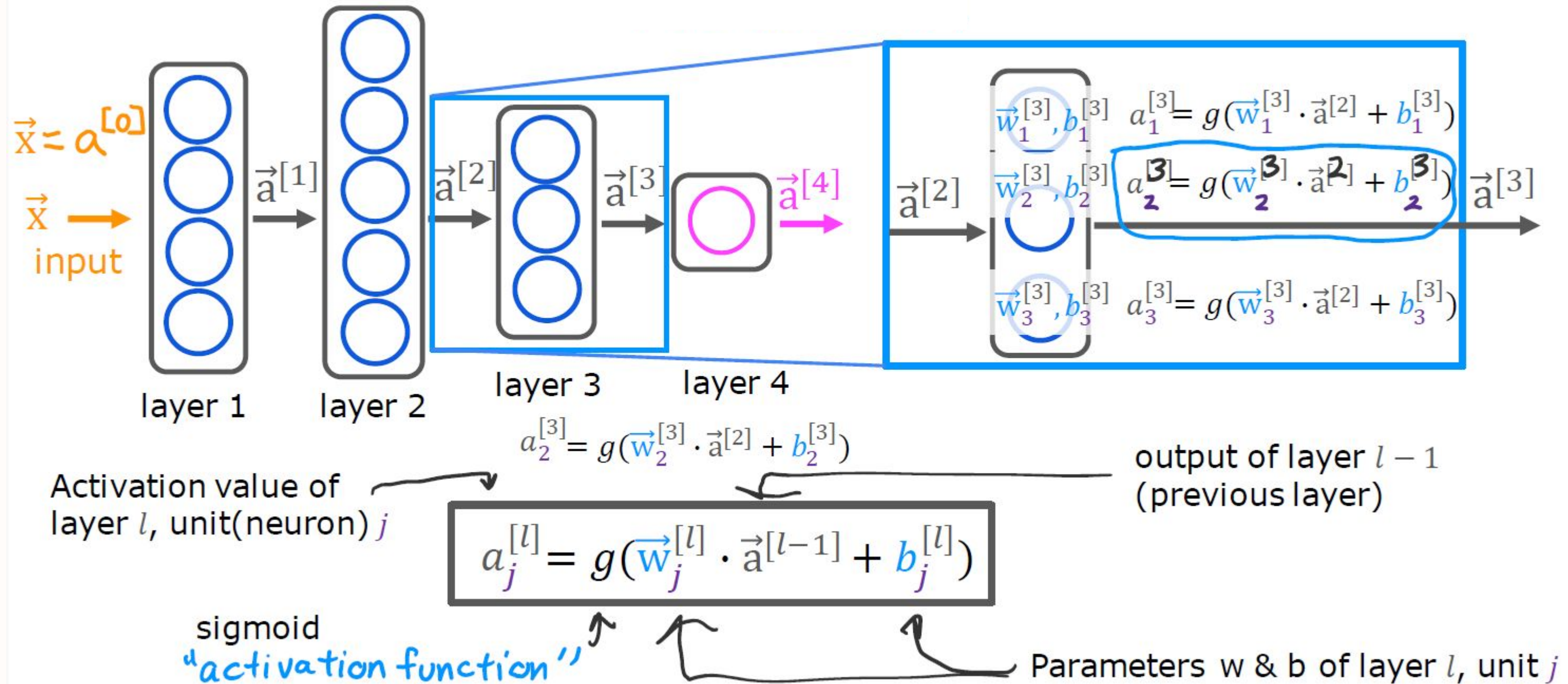
3. More complex neural networks



Question:

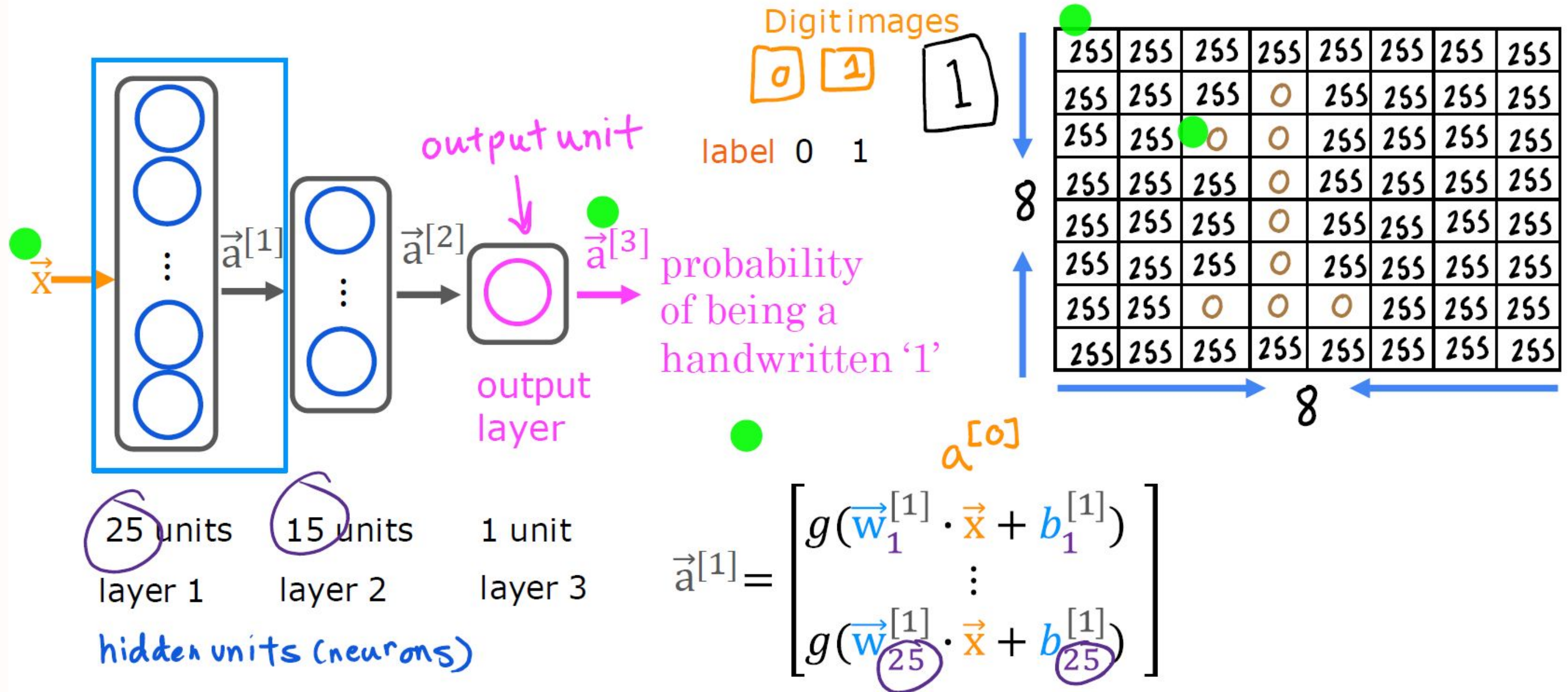
Can you fill in the superscripts and subscripts for the second neuron?

3. More complex neural networks



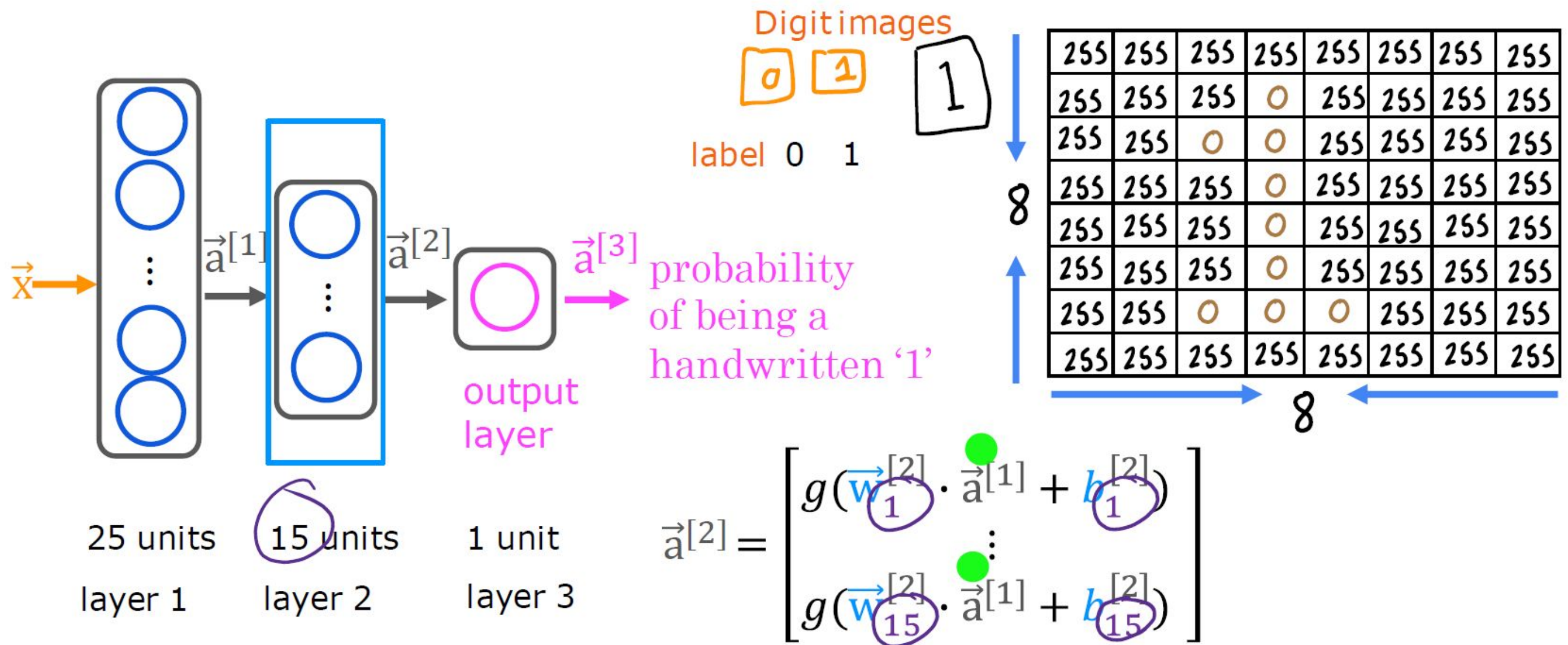
4. Forward propagation - making predictions

Handwritten digit recognition



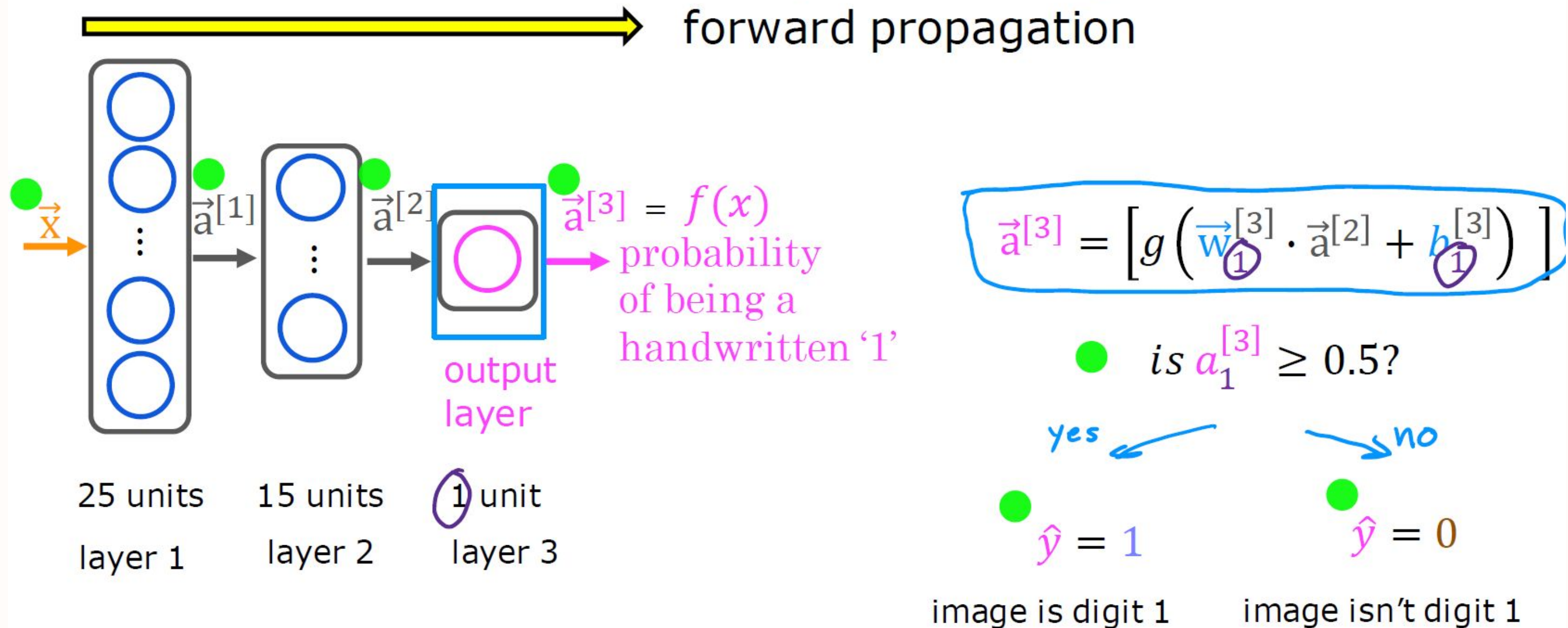
4. Forward propagation - making predictions

Handwritten digit recognition



4. Forward propagation - making predictions

Handwritten digit recognition

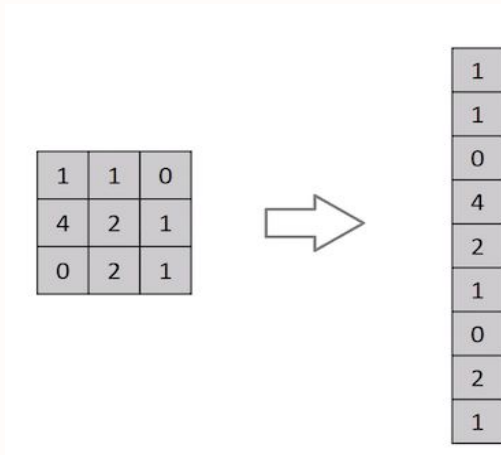


Demo

Convolutional Neural Network

1. Why CNN?

- **Convolutional Neural Network (CNN or ConvNet)** Is class of Deep Neural Network, most commonly applied to analyzing visual imagery.



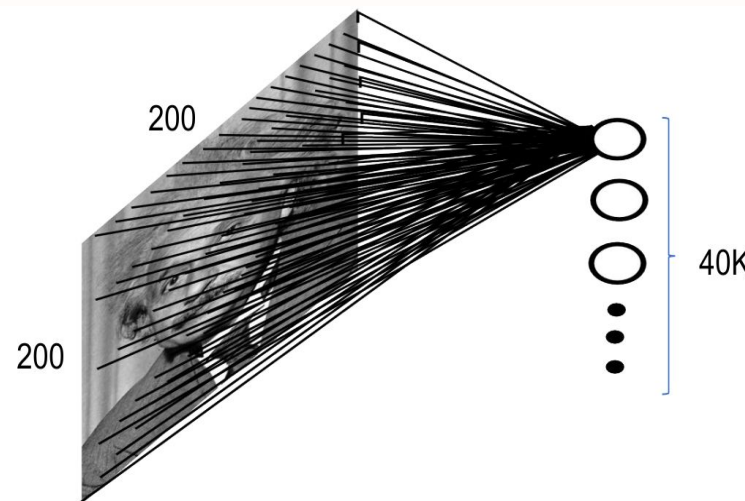
Flatten the image image matrix and feed it to a Multi-Level Perceptron for classification purposes ?



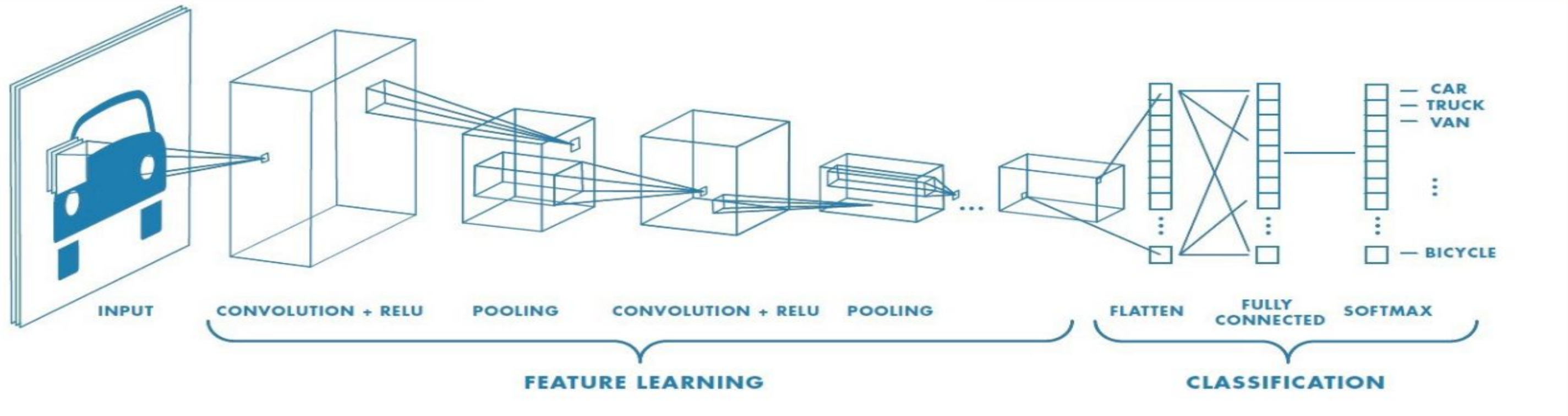
Able to capture the Spatial and Temporal dependencies through the application of relevant filters

Reduce the images into a form which is easier to process, without losing features which are critical for getting a good prediction

Reduce the number of parameter

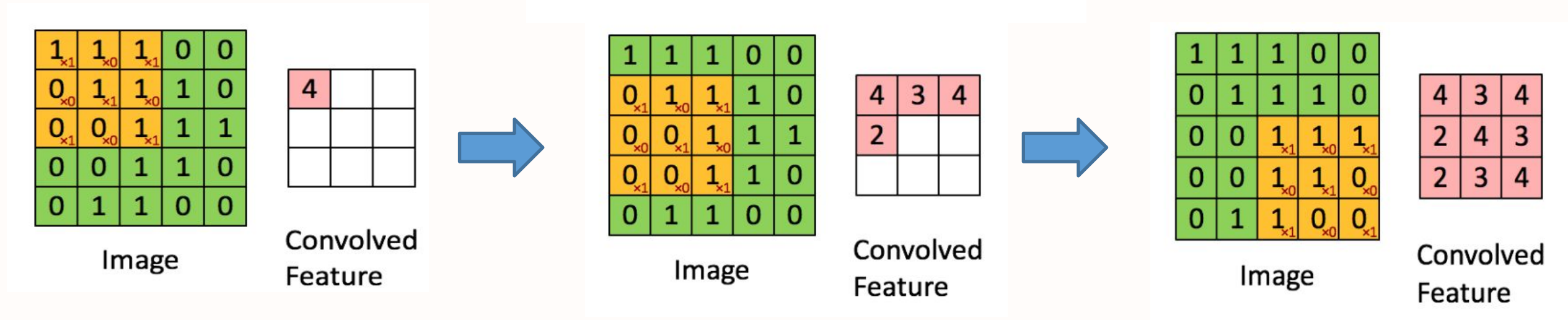


2. CNN architecture



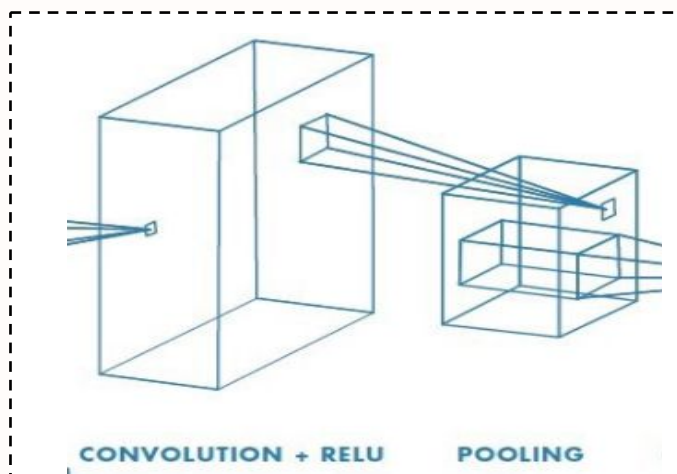
- **Input image** : RGB image (3 channels) or gray scale image (1 channel)
- **Convolutional Layer**: **The learnable kernel** - extract the high-level feature
- **Pooling layer** : decrease the computational power required to process the data
- **Classification** : fully connected layer (FC layer)

3. Convolutional layer



- Parameter
- Kernel size - (3x3), (5x5), (7x7)
- Stride length - 1, 2, ...
- Padding

Note: In CNNs, filters are not defined. The value of each filter is learned during the training process.



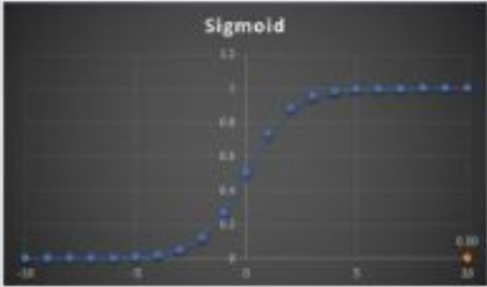
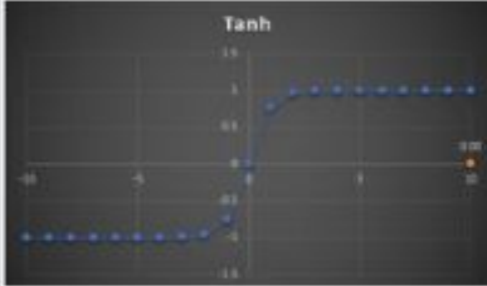
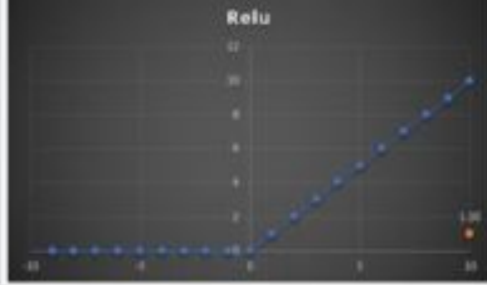
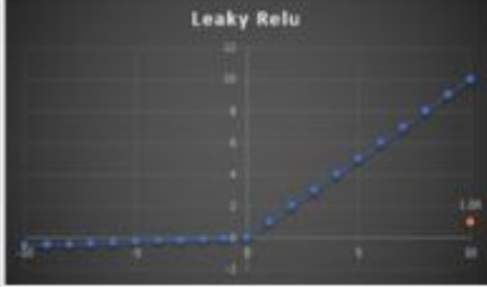
3. Convolutional layer

Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

Operation	Filter	Convolved Image
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

Example about filter/kernel matrix and its effect on the convolved image

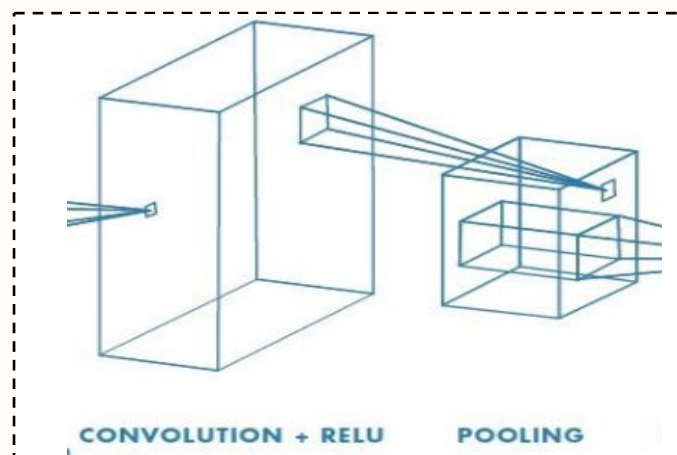
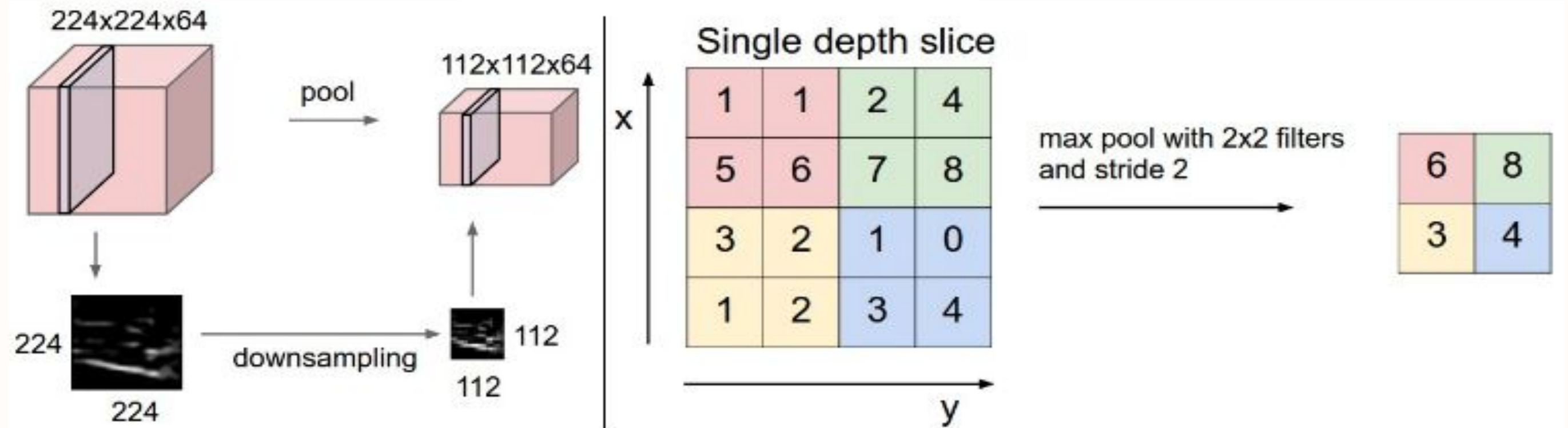
4. Activation function

Name	Plot	Equation	Derivative
Sigmoid		$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
Tanh		$f(x) = \tanh(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$	$f'(x) = 1 - f(x)^2$
Rectified Linear Unit (relu)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Leaky Rectified Linear Unit (Leaky relu)		$f(x) = \begin{cases} 0.01x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0.01 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

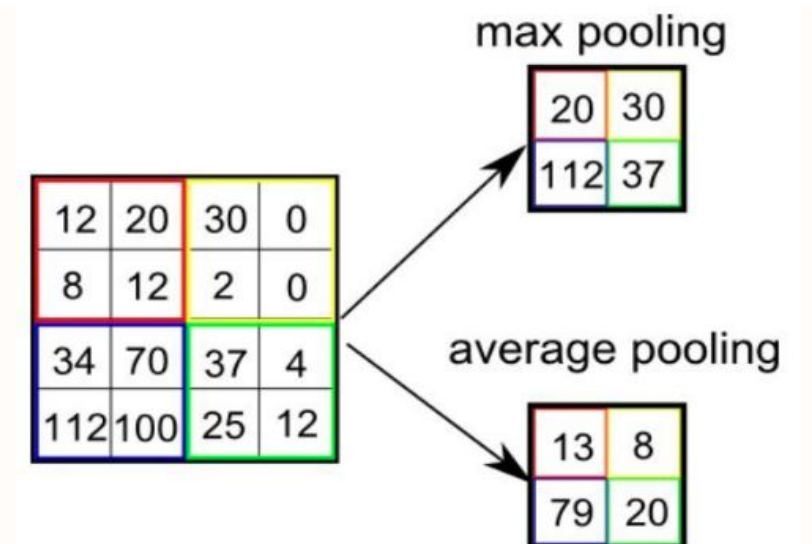
4. Activation function

Function	Advantages	Disadvantages
Sigmoid	1) Smooth Gradient => Preventing “jump” in output value 2) Output values bound between 0 and 1 => Normalizing the output of each neuron 3) Clear prediction	1) Vanishing Gradient 2) Computationally expensive
Tanh	All of Sigmoid’s advantages	1) Vanishing Gradient 2) Computationally expensive
ReLU	Computationally efficient	Dying ReLU problem
Leaky ReLU	Try to solve Dying ReLU problem	Inconsistent results

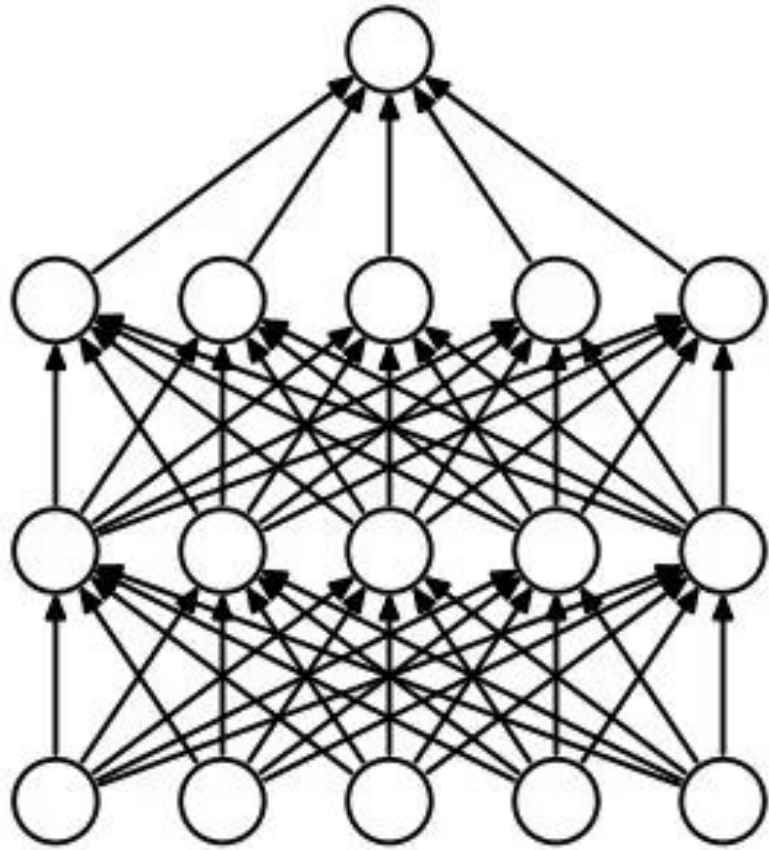
5. Pooling layer



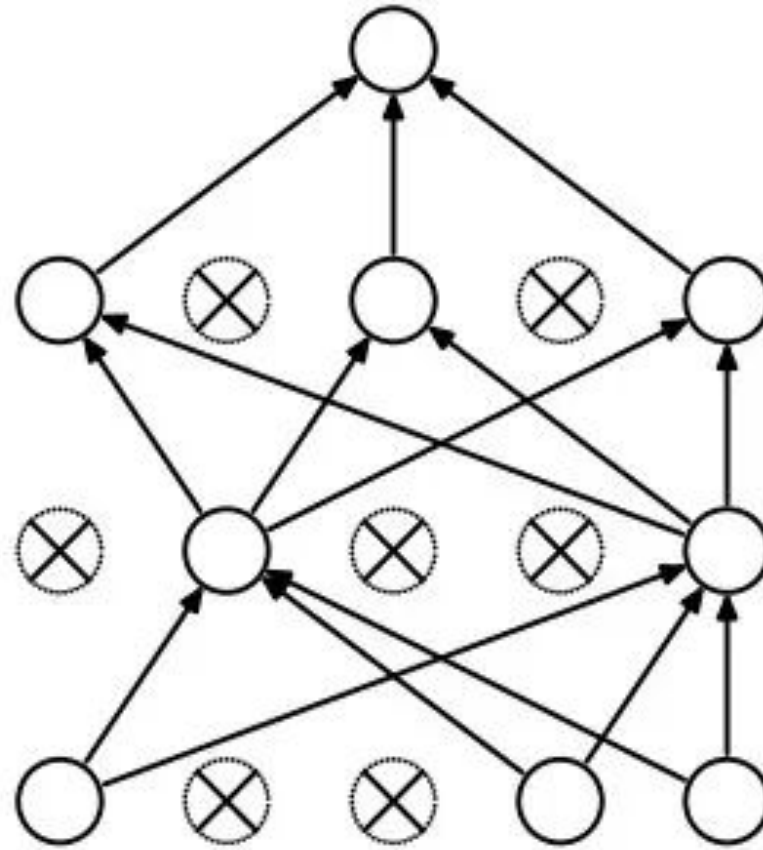
others type pooling



6. Dropout layer



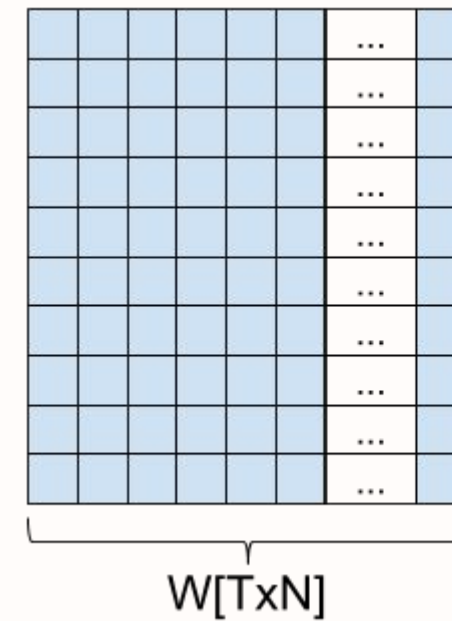
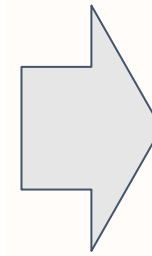
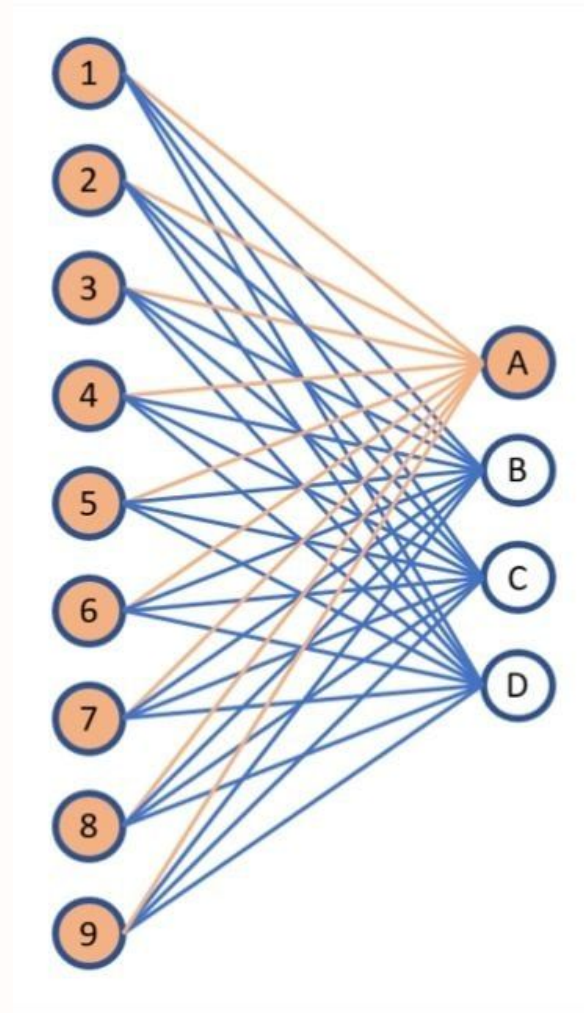
(a) Standard Neural Net



(b) After applying dropout.

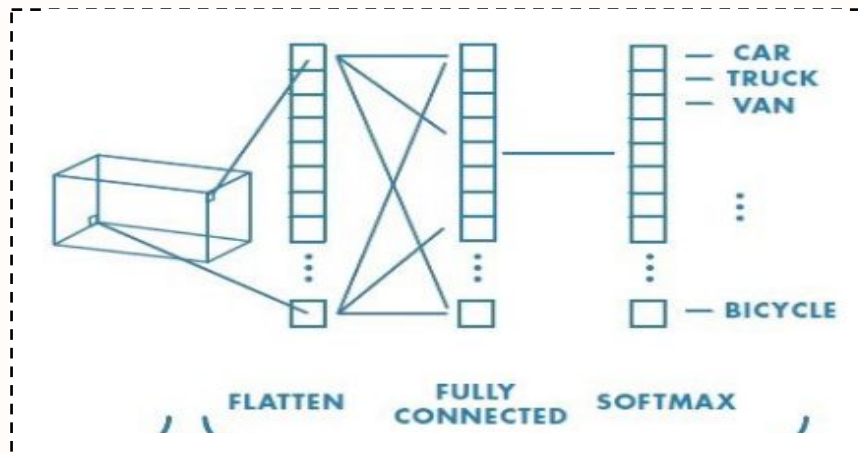
**solve the Overfitting
problem?**

7. Fully connected layer

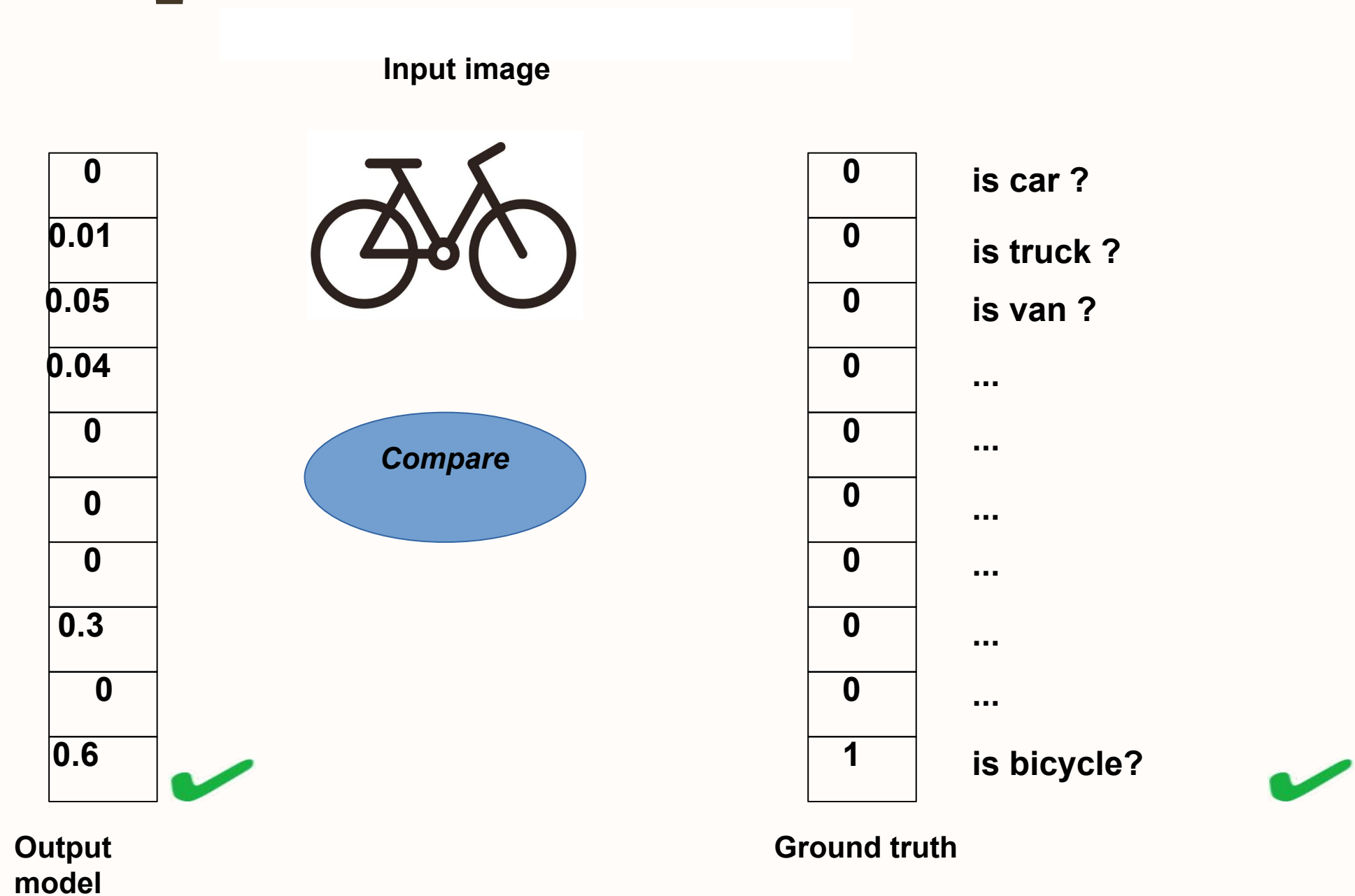


$$\begin{matrix} \text{[Yellow vector]} \\ x[N \times 1] \end{matrix} \cdot \begin{matrix} \text{[Blue vector]} \\ b[T \times 1] \end{matrix} = \begin{matrix} \text{[Pink vector]} \\ y[T \times 1] \end{matrix}$$

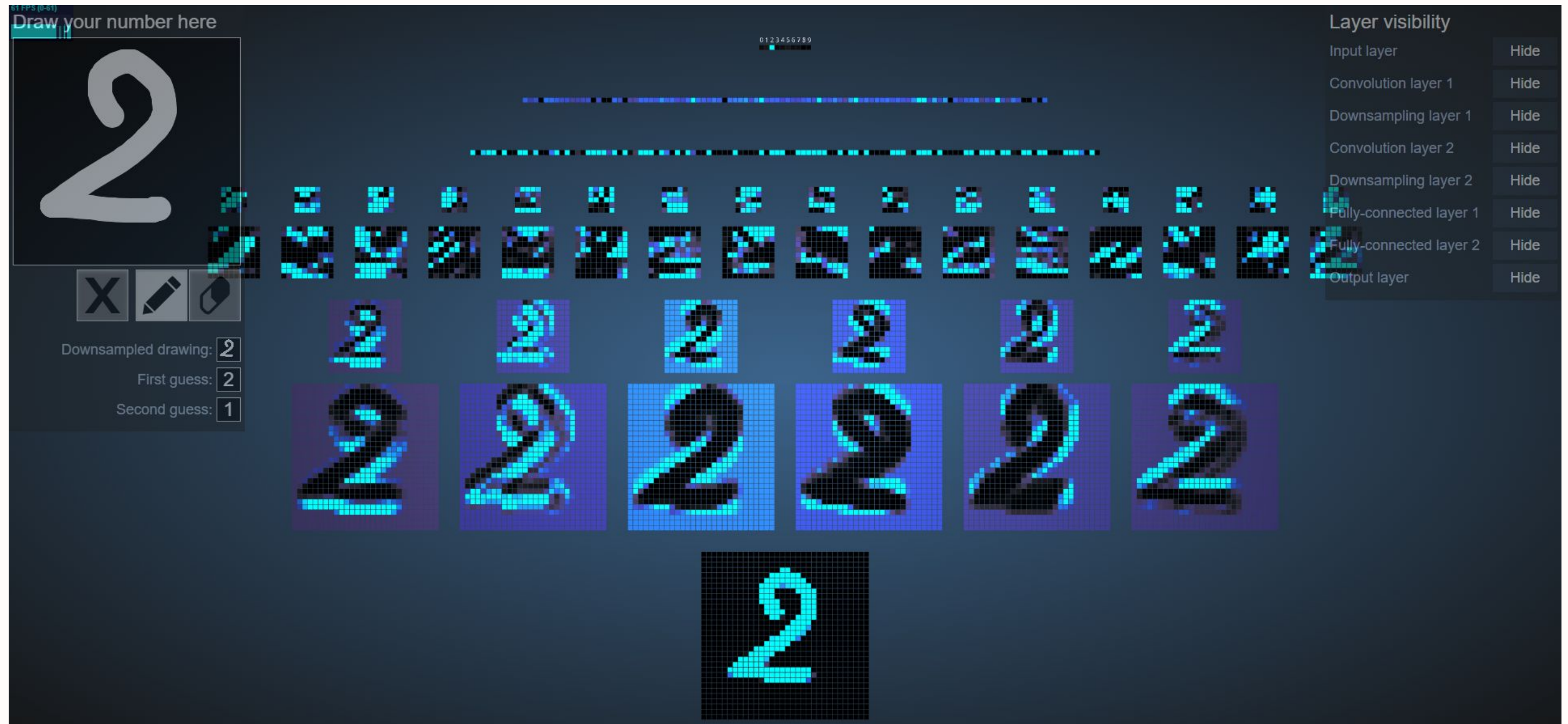
The equation shows the matrix multiplication of the input vector x (yellow) with the weight matrix W (blue grid), followed by adding the bias vector b (blue) to produce the output vector y (pink).



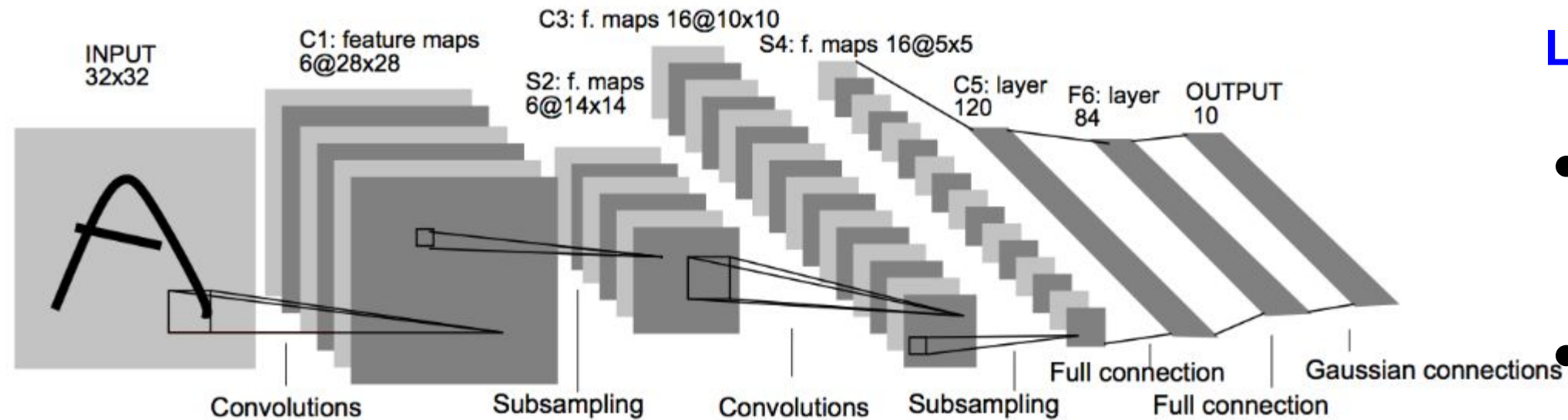
8. Loss and Optimization



9. CCN Visualization Example

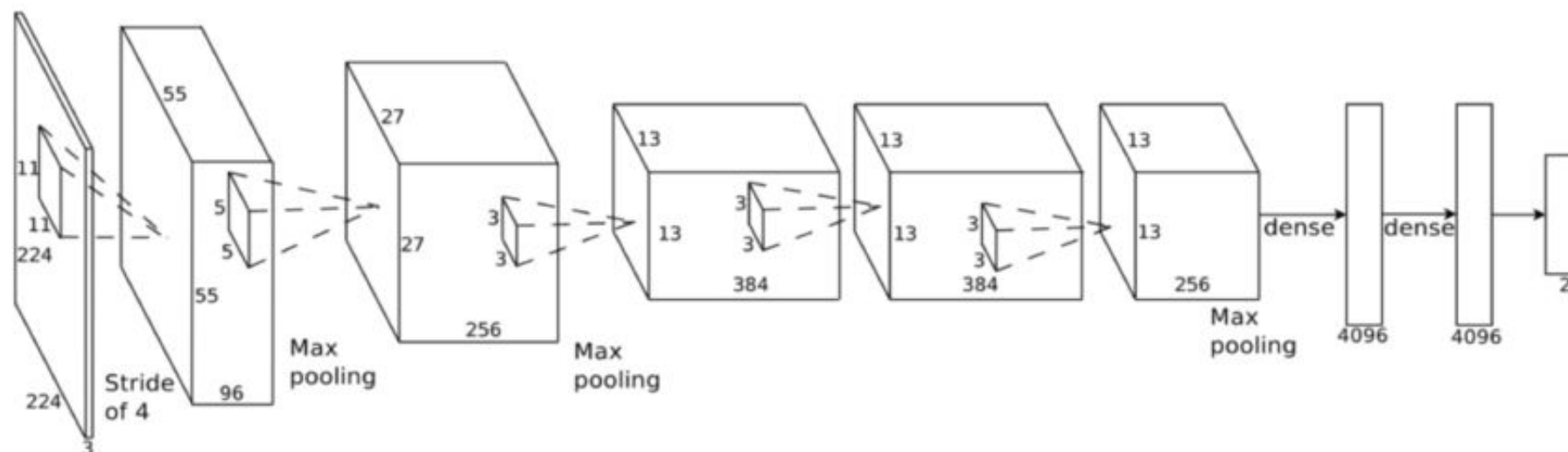


10. CNN typical networks



LeNet

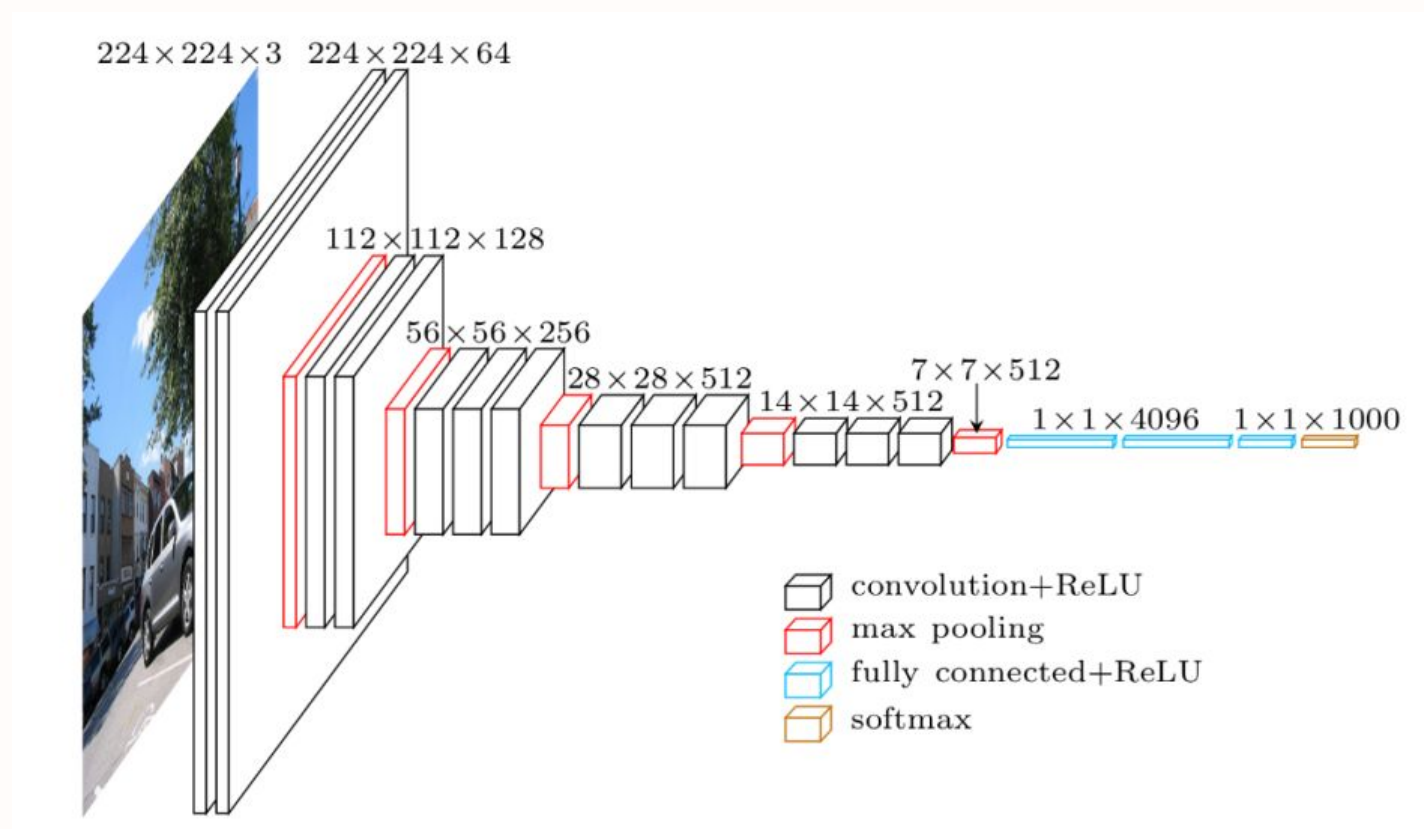
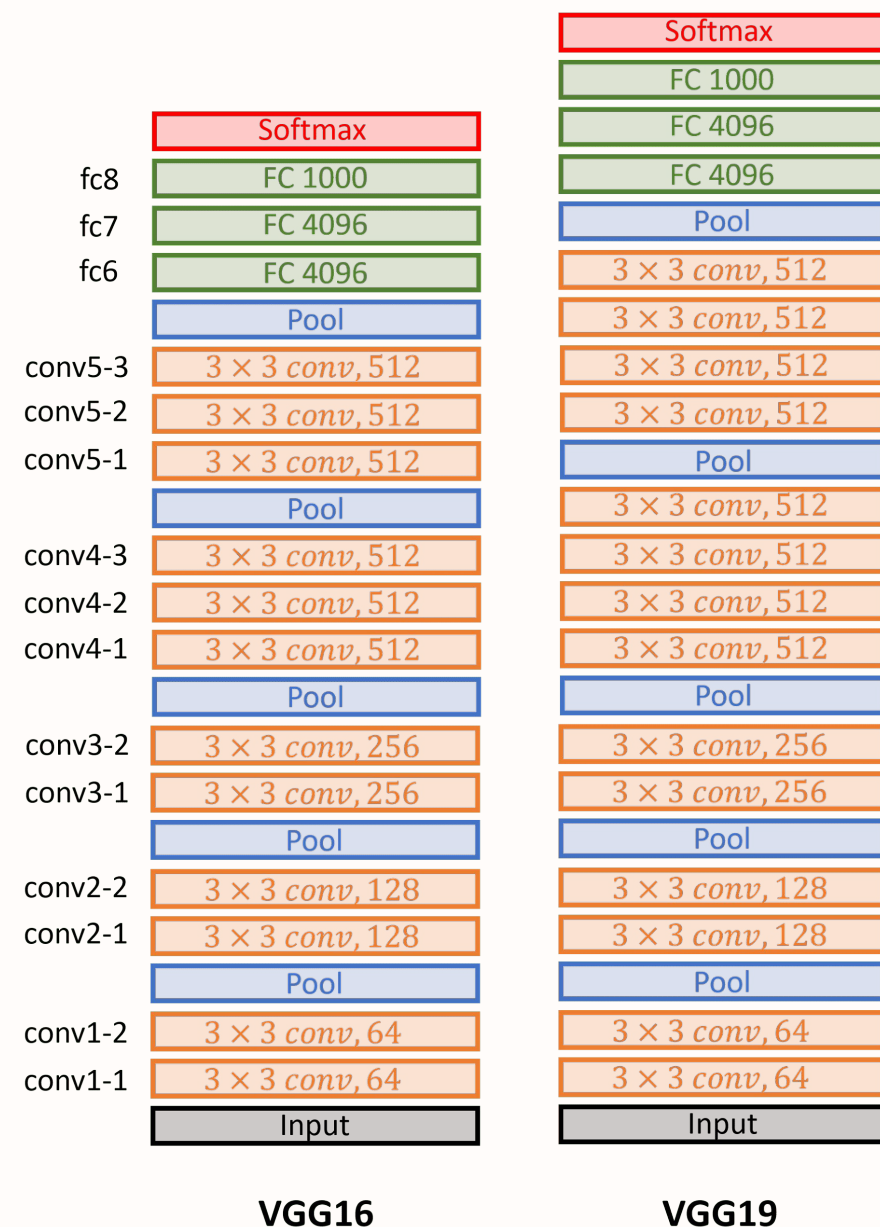
- Developed by Yann LeCun in 1990's
- Parameter: 60,000



AlexNet

- Similar but deeper, bigger
- Parameter: 60 million

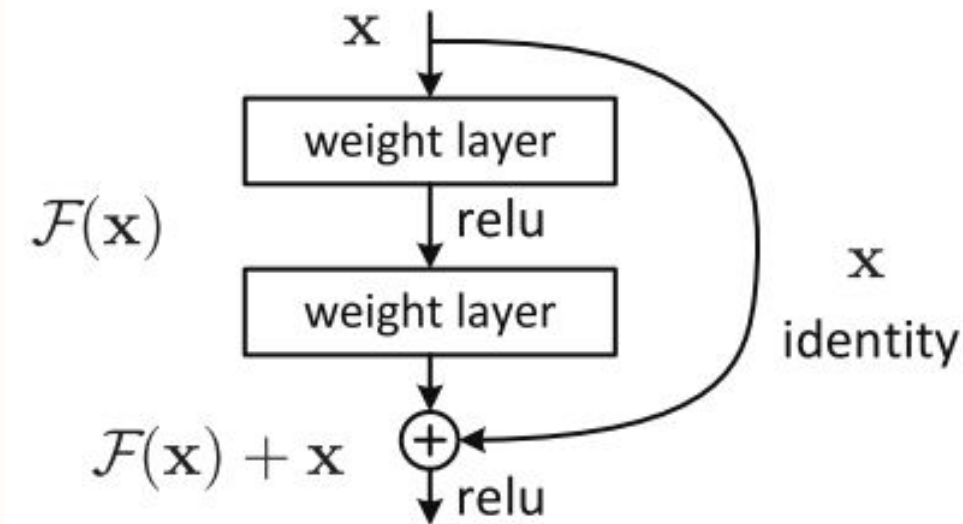
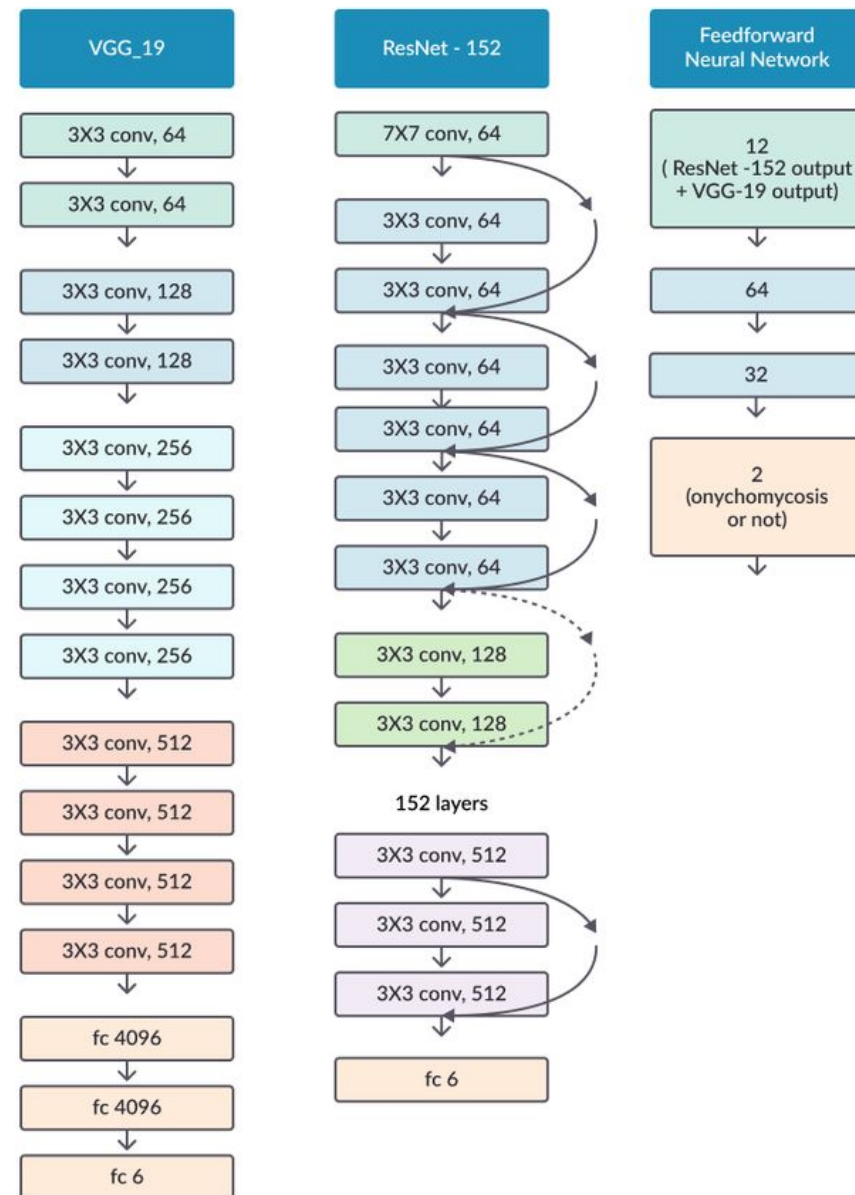
10. CNN typical networks



VGGNet

- Contribution: “showing that the depth of the network is a critical component for good performance”
- Parameter: 138 million

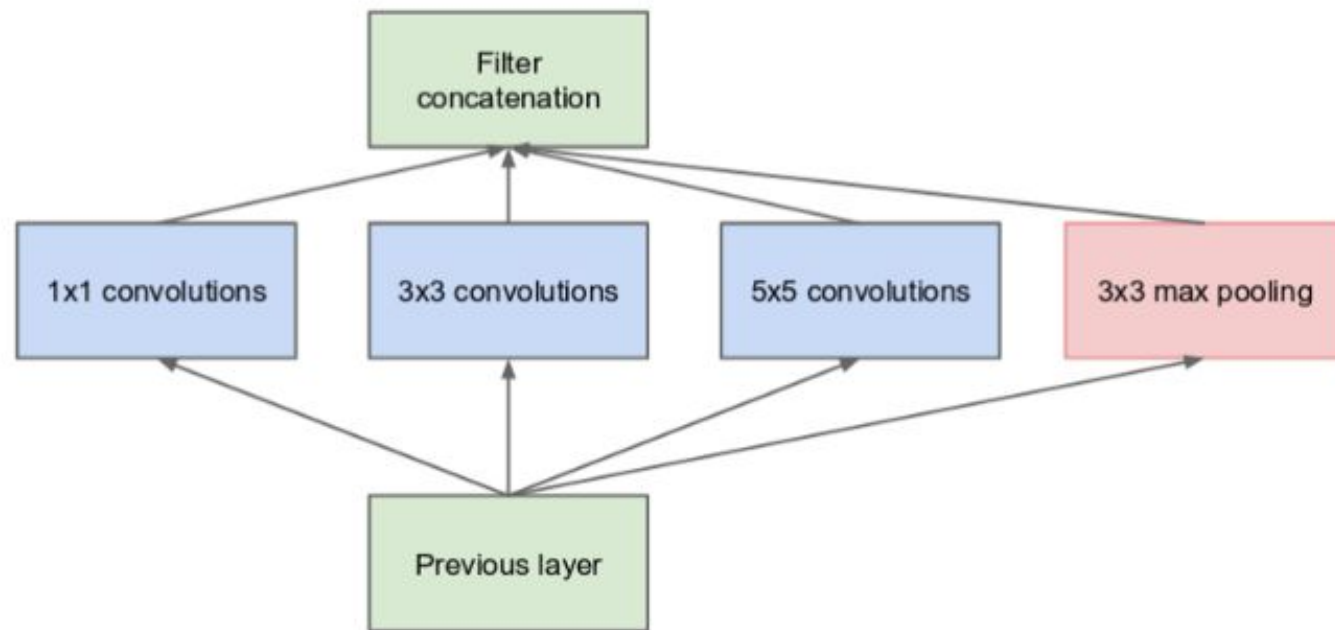
10. CNN typical networks



ResNet

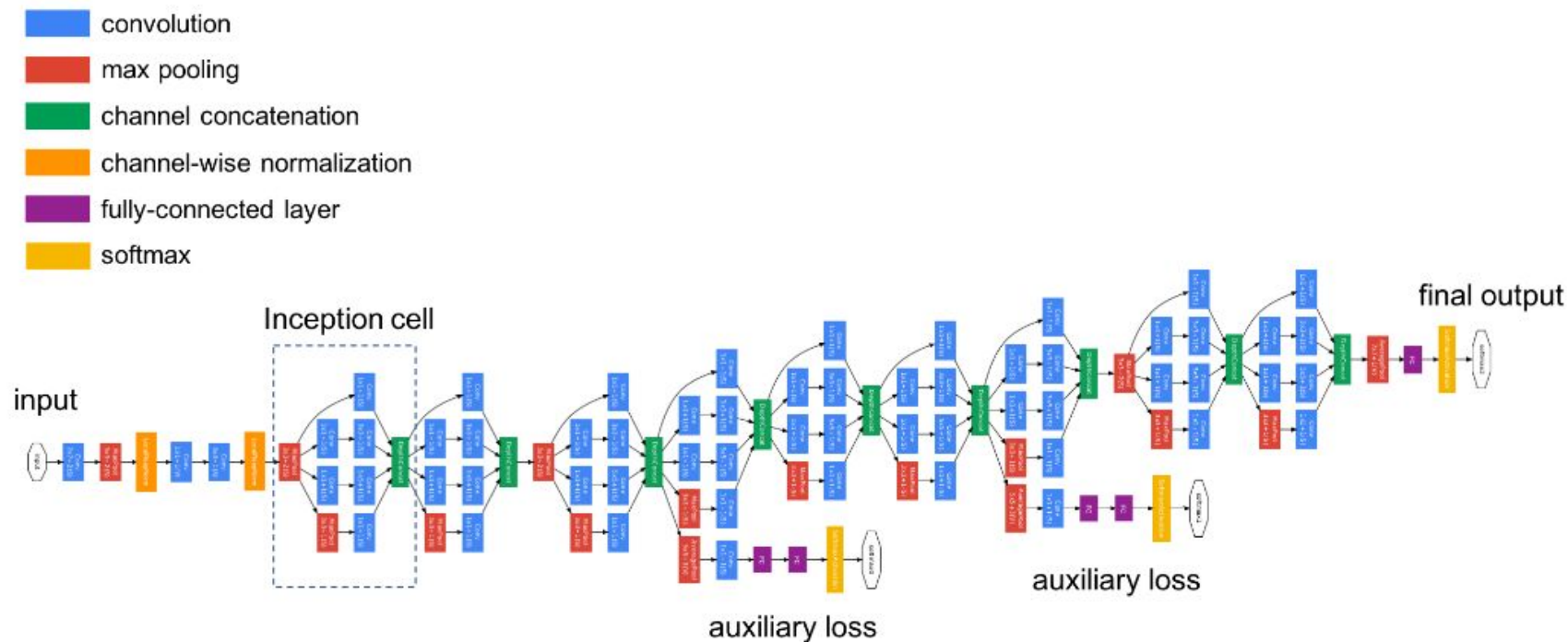
- Skip connections --> Solve Vanishing Gradient
- 25 million (ResNet 50)

10. CNN typical networks



Inception (GoogLeNet)

- Deal with large variation in size
- Parameter: 5 million (V1) and 23 million (V3)

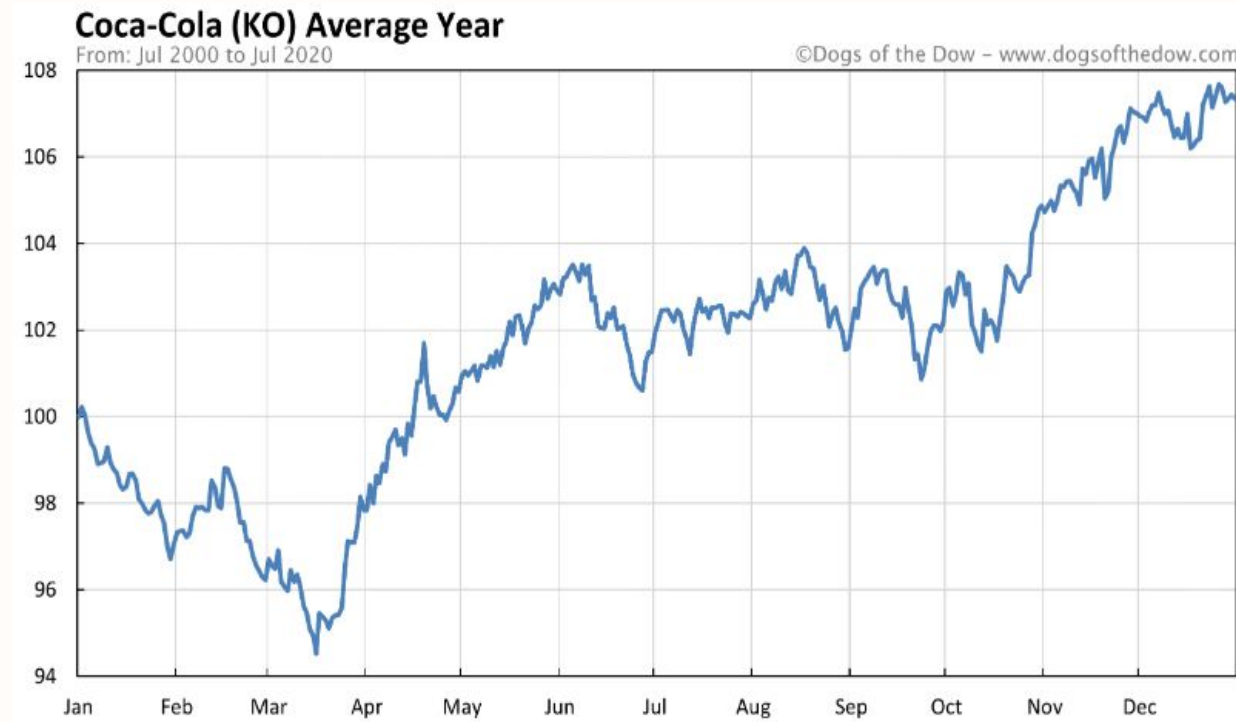


Demo

<https://github.com/vzhou842/cnn-from-scratch/tree/master>

Recurrent Neural Network (RNN)

1. Sequence type data



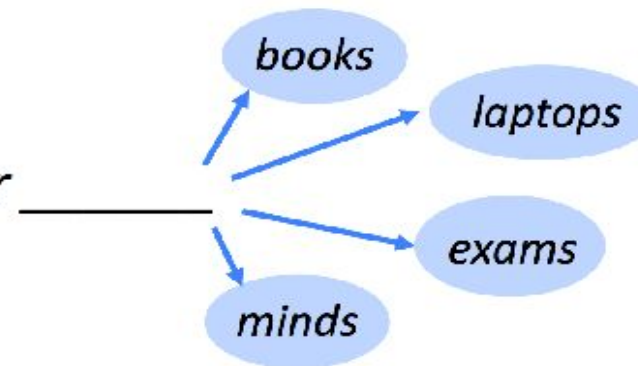
Stocks price



Video data

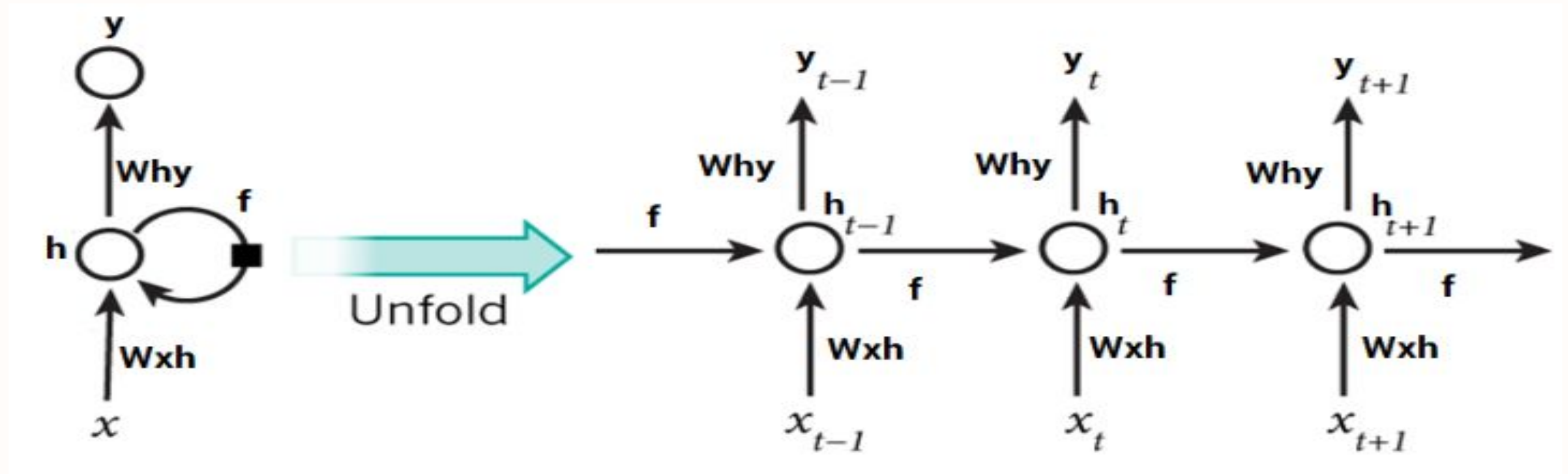
Language
model

the students opened their



$$P(w_m | w_1, \dots, w_{m-1})$$

2. RNN architecture

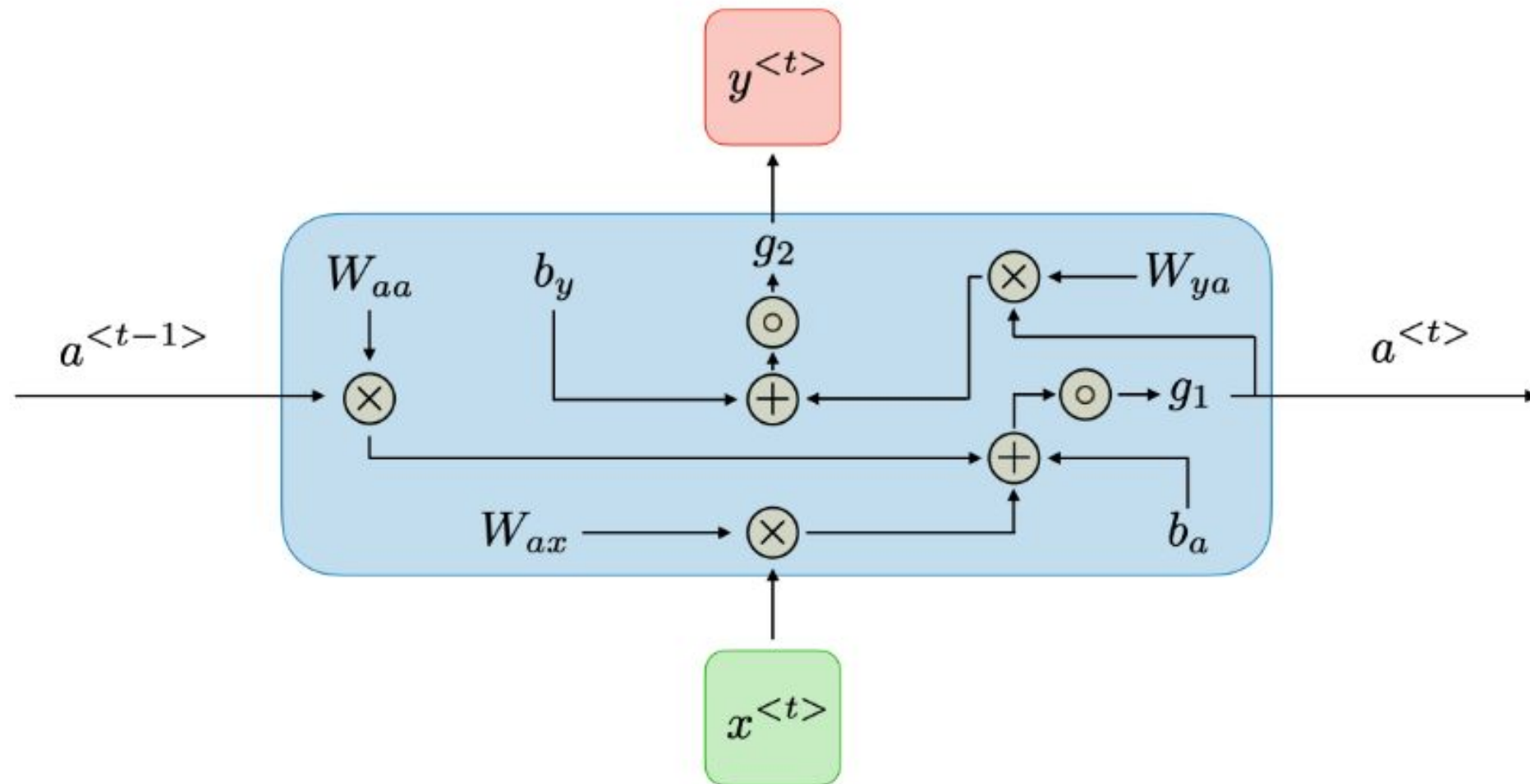


2. RNN architecture

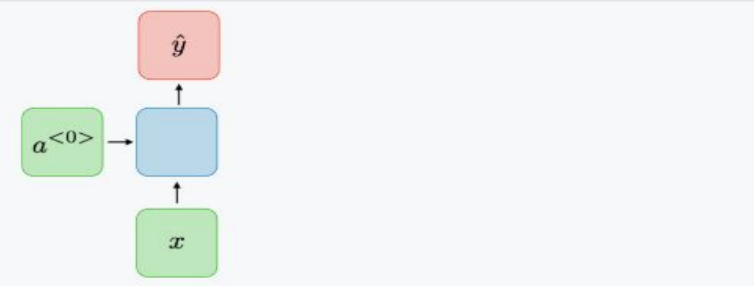
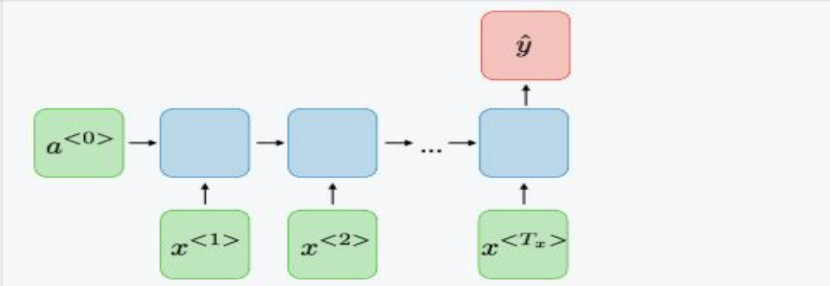
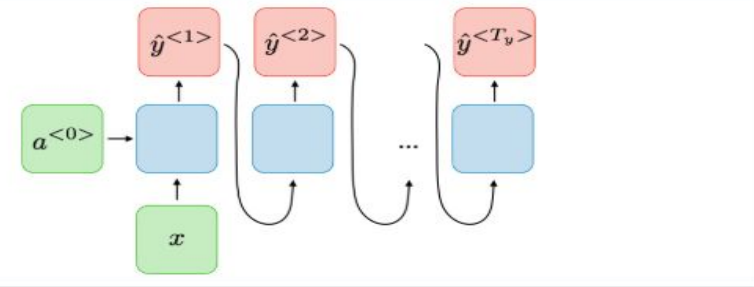
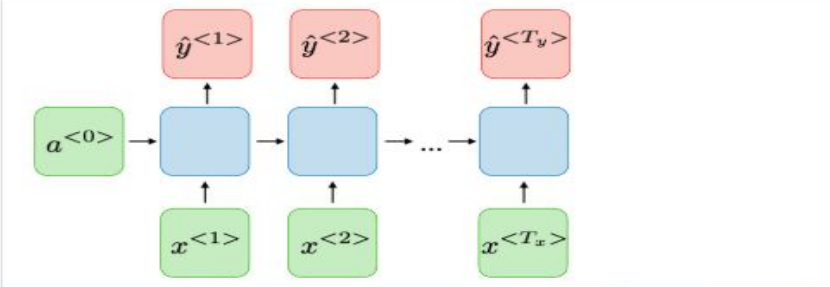
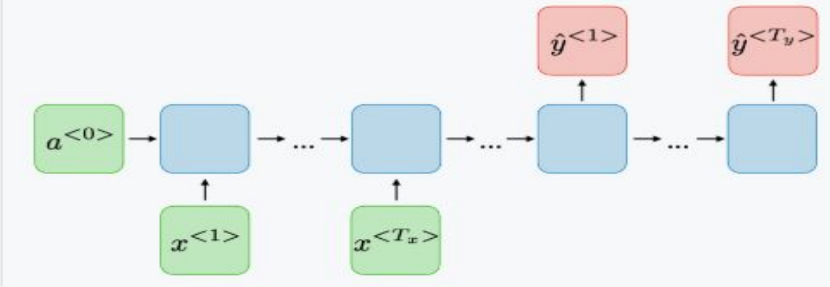
For each timestep t , the activation $a^{<t>}$ and the output $y^{<t>}$ are expressed as follows:

$$a^{<t>} = g_1(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a) \quad \text{and} \quad y^{<t>} = g_2(W_{ya}a^{<t>} + b_y)$$

where $W_{ax}, W_{aa}, W_{ya}, b_a, b_y$ are coefficients that are shared temporally and g_1, g_2 activation functions.

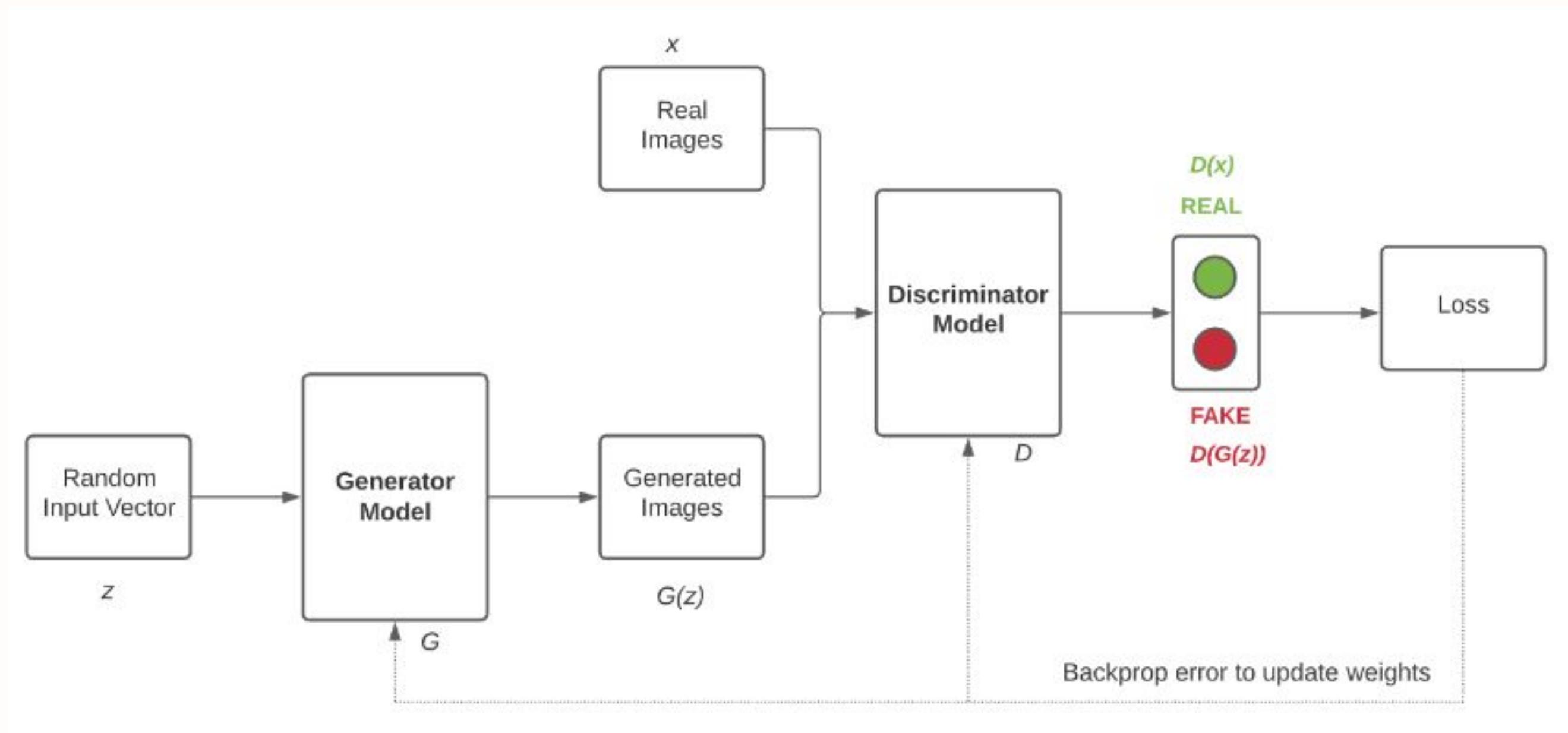


3. Different types of RNN

One-to-one $T_x = T_y = 1$		Traditional neural network	Many-to-one $T_x > 1, T_y = 1$		Sentiment classification
One-to-many $T_x = 1, T_y > 1$		Music generation	Many-to-many $T_x = T_y$		Name entity recognition
			Many-to-many $T_x \neq T_y$		Machine translation

Generative Adversarial Networks (GAN)

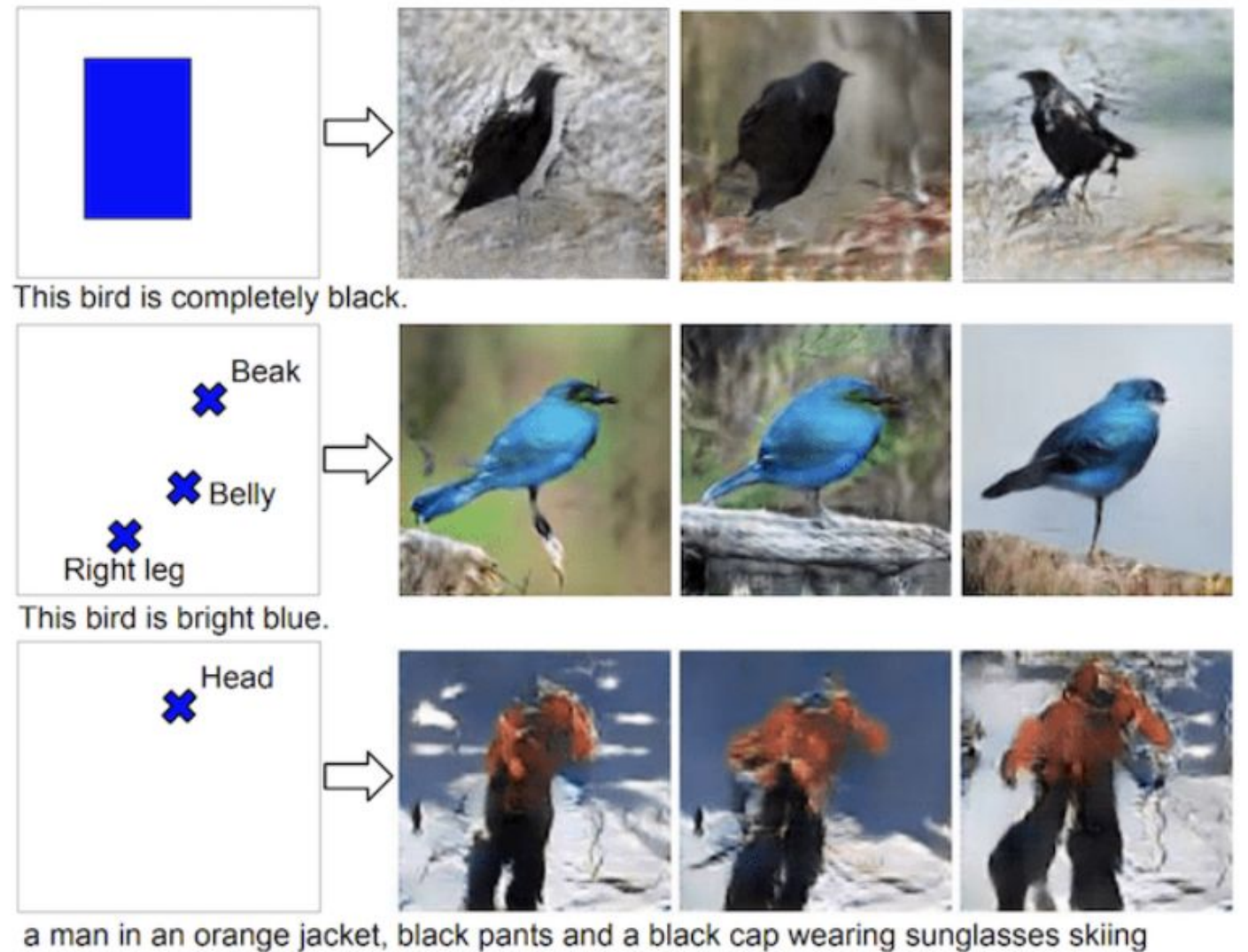
1. GAN Architecture



2. Application



Image to image



Text to image